GUIA INTEGRADO · ATUALIZAÇÃO 02 NOV 2026

BACKEND JAVA + SPRING DO ZERO AO AVANÇADO

Plano completo com duas trilhas, provas diagnósticas, mecanismos internos, projetos reais e critérios de competência.

Java 25 LTS · Spring Boot 4.1.2 · Spring Framework 7 · PostgreSQL · Spring Security · Arquitetura

Trilha de 24 meses + opção acelerada de 12 meses

Revisão: 02 NOV 2026

NAVEGAÇÃO DO GUIA

Sumário

Versão integrada: plano tradicional e alternativa acelerada com critérios de entrada.

Início e diagnóstico 3–6
Objetivos, testes de entrada e baseline

Método e trilhas 7–14
Rotina, roadmap e plano acelerado

Plano de 24 meses 16–23
Trimestres, entregas e checkpoints

Módulos e aprofundamentos 24–51
17 módulos + mecanismos e laboratórios

Arranque e projetos 52–62
8 semanas e projetos evolutivos

Exercícios e laboratórios 63–78
Código, segurança, testes e confiabilidade

Avaliação, arquitetura e carreira 80–85
Gates, arquitetura e entrevistas

Referências e próximos passos 86–94
37 referências originais + livros e links

Como começar: aplique o diagnóstico de entrada e use os gates para definir o percurso. Números de página referem-se às seções deste arquivo.

INTRODUÇÃO

O que você vai construir

Um plano de competência, não uma coleção de checklists.

Este guia foi pensado para formar uma base forte em **engenharia backend com Java** e evoluir, com projetos progressivos, até Spring Boot, arquitetura de aplicações e sistemas distribuídos.

> **Conceito-chave.** Java é a **linguagem**. Spring Framework é a base de infraestrutura e inversão de controle; Spring Boot acelera a criação, configuração e operação de aplicações Spring.

Resultados esperados

- Escrever programas Java com clareza, testes e boa modelagem orientada a objetos.
- Criar APIs HTTP robustas com Spring MVC e contratos bem definidos.
- Modelar e consultar dados em PostgreSQL; controlar transações com consciência.
- Aplicar autenticação, autorização, validação e medidas de segurança.
- Implantar aplicações com Docker e CI/CD, utilizando logs, métricas e tracing.
- Projetar sistemas e justificar decisões com requisitos e medições.

Recomendação de stack

Camada	Escolha principal	Quando utilizar
Linguagem	Java 25 LTS	Projetos novos; usar 21 LTS em exercícios que dependam de material antigo.
Framework	Spring Boot 4.1.2	Referência estável em 02/11/2026; valide patches no Initializr.
Build	Maven	Aprender estrutura de dependências, perfis e ciclo de build.
Persistência	PostgreSQL + JDBC + JPA	Aprender SQL antes de automatizar o acesso com ORM.
Testes	JUnit + Mockito + Testcontainers	Testar lógica, integração real e contratos.
Entrega	Docker + GitHub Actions	Empacotar, testar e implantar com repetibilidade.

> **Atenção à versão:** as APIs e starters mudam. Na série Boot 4, `spring-boot-starter-webmvc` substitui a recomendação antiga de `spring-boot-starter-web`. Verifique o Spring Initializr e a documentação antes de copiar exemplos.

DIAGNÓSTICO E ROTEAMENTO

Qual trilha é a sua?

A avaliação inicial determina o caminho; não presuma conhecimento intermediário apenas porque o complemento o menciona.

Trilha completa • 24 meses

Para quem ainda precisa consolidar Java, Git, SQL, HTTP e testes. Siga os meses 1 a 24 e os projetos originais.

10–15 h/semana

Trilha acelerada • 12 meses

Para quem demonstrar domínio nos checkpoints iniciais. Substitui revisão dominada por Java moderno, internals, segurança e arquitetura.

~20 h/semana

Procedimento obrigatório

Etapa	Ação	Regra de decisão
1. Linha de base	Provas M3, M6 e M9, com cronômetro; nenhum tutorial ou IA.	Guarde código, testes e tempo.
2. Pontuação	Compreensão, implementação, debugging e validação, de 0 a 4.	Todas as dimensões ≥ 3 para dispensar revisão.
3. Lacunas	Conteúdos com nota 0–2 viram sprints de recuperação.	Não pule o gate correspondente.
4. Escolha	Se houver lacunas extensas, siga a trilha de 24 meses.	12 meses é alternativa, não obrigação.

> **Interpretando o arquivo complementar.** Ele descreve um cenário de desenvolvedor intermediário com experiência em SaaS; isso é uma hipótese de entrada, não um dado comprovado sobre você. Este plano valida a hipótese antes de acelerar.

Como registrar a evidência

Crie a pasta `diagnostico/` no seu repositório de estudos, com `java/`, `sql/`, `spring/` e `retrospectiva.md`. Preserve versões antes de receber feedback.

GATES INICIAIS

Prova diagnóstica · 3, 6 e 9

Três desafios curtos, progressivos e sem tutorial. São provas de autonomia, não de memorização.

Prova D1 · Java (90 minutos)

Implemente um livro-caixa que recebe eventos `Entrada` e `Saida` e totaliza o saldo usando `BigDecimal`.

- Rejeitar valor nulo e negativo; não usar `double` para dinheiro.
- Utilizar classes imutáveis ou records; adicionar testes para ordem dos eventos, zero e saldo insuficiente.
- Explicar estrutura de dados, `equals`/`hashCode` e tratamento de falhas.

Prova D2 · SQL + HTTP (100 minutos)

Modele `clientes`, `reservas` e `recursos`; produza DDL com PK/FK, restrição útil e três queries com JOIN.

- Explique quando retornar 400, 401, 403, 404 e 409 em uma API de reservas.
- Escreva uma transação de criação de reserva e descreva como impedir duplicidade concorrente.
- Mostre um plano de execução e proponha, com evidência, um índice.

Prova D3 · Spring Core (120 minutos)

Crie um endpoint de reservas usando injeção por construtor; um bean só deve existir quando uma propriedade estiver habilitada.

- Mostre com teste de contexto a presença e ausência do bean.
- Teste um caso HTTP válido e um inválido com MockMvc.
- Explique por que uma chamada interna entre métodos do mesmo bean pode contornar o proxy transacional.

> **Avaliação:** evidência executável + explicação + testes, com nota 0–4 por eixo. Se qualquer eixo ficar abaixo de 3, revise apenas os tópicos relevantes e refaça a prova com outro enunciado.

BASELINE TÉCNICA

Versões e migração · novembro/2026

Fixe uma baseline reproduzível e confirme compatibilidades antes de copiar dependências de artigos.

> **Referência verificada em 02/11/2026:** Java 25 LTS e Spring Boot **4.1.2**, lançado em 23/10/2026. Spring Boot 4.2.0-RC1 é pré-lançamento, não baseline estável neste documento.

Assunto	Regra prática
Spring Boot	Use 4.1.2 como snapshot editorial; em projeto novo confirme o patch estável no Initializr.
Java	Java 25 LTS; recursos preview somente com flags explícitas e sem tratá-los como estáveis.
Starters	`spring-boot-starter-webmvc` e `spring-boot-starter-webmvc-test`; não começar pelo starter web depreciado.
Flyway	`spring-boot-starter-flyway` + `flyway-database-postgresql` para PostgreSQL.
Mocks	`@MockitoBean` / `@MockitoSpyBean` substituem mocks do Boot removidos.
Testes MVC	`@SpringBootTest` + `@AutoConfigureMockMvc`, ou slice MVC.
Cliente HTTP	Aprenda `RestClient` + HTTP Interfaces e configure timeouts/testes.
JSON	Jackson 3: mudança de pacotes; anotações Jackson ainda usam `com.fasterxml.jackson.annotation`.

Comandos de checagem

```
java -version
./mvnw -version       # macOS/Linux
.\mvnw.cmd -version  # Windows PowerShell
.\mvnw.cmd dependency:tree
```

> Não fixe como verdade imutável datas de suporte, versão máxima do JDK ou suporte de bibliotecas. Consulte requisitos e matriz de suporte da **versão exata** em uso. Para upgrades, leia notas de migração e rode testes.

MÉTODO

Como utilizar este guia

Aprendizagem deliberada, evidências de domínio e feedback.

Ciclo de aprendizagem

- Compreenda o conceito e escreva um resumo curto.
- Resolva um exercício sozinho, sem copiar código.
- Teste o resultado e investigue cenários-limite.
- Refatore, documente e publique a evolução no Git.
- Explique quando e por que utilizaria a técnica.

Regra de progressão

- Não avance porque um vídeo terminou.
- Faça uma entrega executável em cada módulo.
- Peça revisão crítica e teste comportamentos.
- Se falhar no critério de saída, revise o tema.

Legenda das atividades

Tipo	Como praticar
E - Exercício	Problema pequeno, 15 a 60 minutos, sem solução pronta.
L - Laboratório	Experimentação controlada para investigar um conceito.
P - Projeto	Entrega integrada com requisitos, testes e README.
G - Gate	Avaliação prática obrigatória antes de avançar.

Autonomia e inteligência artificial

- Tente sozinho por 20 a 40 minutos antes de pedir uma implementação.
- Peça pistas, revisão de design, explicação de erros e casos de teste, não só a resposta.
- Antes de aceitar uma sugestão, explique o que cada trecho faz e seus trade-offs.
- No projeto final, documente explicitamente o que você validou por conta própria.

> **Métrica principal:** conseguir implementar, testar, explicar e modificar uma solução **sem roteiro passo a passo**. A quantidade de horas ou de certificados não substitui essa evidência.

Ritmo recomendado

Use 10–15 horas semanais como referência. Com 6–8 horas, alongue o cronograma. Com 20 horas, aprofunde exercícios e projetos antes de acelerar. Os 24 meses são uma estimativa curricular, não uma promessa de senioridade.

CONFIGURAÇÃO

Preparação do ambiente

Ferramentas essenciais e verificações reproduzíveis.

Antes de iniciar o Spring, deixe Java, terminal, Git, IDE e banco funcionando isoladamente.

Ferramenta	Configuração sugerida
JDK	Eclipse Temurin 25 LTS; Java 21 LTS para materiais e projetos que ainda o exigem.
IDE	IntelliJ IDEA Community, Eclipse ou VS Code com extensões Java.
Terminal	PowerShell e comandos essenciais Linux (WSL opcional).
Build	Maven Wrapper do projeto; evite depender apenas de Maven global.
Banco	PostgreSQL local ou ambiente Docker; cliente psql, DBeaver ou pgAdmin.
API	curl ou Bruno/Postman para exercitar requisições.
Versionamento	Git + repositório remoto no GitHub.
Containers	Docker Desktop/Engine apenas quando chegar à fase de infraestrutura.

Checklist básico (PowerShell)

```
java -version
javac -version
git --version
# Dentro de um projeto Spring com Maven Wrapper:
.\mvnw.cmd --version
.\mvnw.cmd test
.\mvnw.cmd spring-boot:run
```

Se os comandos falharem: valide o PATH, JAVA_HOME e a instalação do JDK. Para o primeiro contato com Spring, gere o projeto em start.spring.io, com Java, Maven e uma versão estável do Boot.

Primeira aplicação Spring (mais tarde, no mês 9)

Dependências comuns: `Spring Web MVC`, `Validation`, `Spring Data JPA`, driver PostgreSQL; acrescente `Spring Security` e testes de integração quando o projeto exigir. Evite instalar muitas dependências sem saber seu propósito.

Máquina com recursos limitados: comece com JVM + PostgreSQL e uma IDE. Use containers e ferramentas adicionais gradualmente; executar todas as ferramentas simultaneamente não é necessário para aprender.

ROTINA

Sua rotina semanal de estudo

Consistência e revisão são componentes do currículo.

Modelo de 12 horas por semana

Bloco	Horas	Método de execução
Teoria e leitura	2 h	Leia documentação e produza anotações com exemplos próprios.
Programação de projetos	6 h	Implemente uma funcionalidade a partir de critérios de aceitação.
Algoritmos e problemas	2 h	Resolva 2–4 problemas, depois revise a complexidade.
Testes e revisão	2 h	Escreva testes, refatore, abra PR ou registre aprendizados.

Ritual de cada sessão (90 minutos)

Intervalo	Atividade
00–10 min	Revisão ativa: explicar o tema anterior sem consultar notas.
10–30 min	Um conceito novo com documentação e um exemplo mínimo.
30–75 min	Exercício ou implementação do projeto principal.
75–90 min	Testes, commit e registro do que ficou incompleto.

Revisão espaçada

- Revisite temas em 1, 7 e 30 dias.
- Refaça um exercício antigo.
- Compare a primeira versão com a refatorada.

Caderno de engenharia

- Erro observado e hipótese.
- Experimento e resultado.
- Decisão tomada e motivo.
- O que investigar depois.

Checklist de uma entrega de qualidade

- O projeto inicia por instruções documentadas no README.
- Possui exemplos reais de entrada/saída e mensagens de erro úteis.
- Inclui testes relevantes e casos inválidos.
- Usa commits com intenção clara e não contém credenciais.
- Você consegue explicar o fluxo de dados de ponta a ponta.

Sinal de alerta: se você passa a maior parte do tempo ajustando ferramentas em vez de programar, reduza o stack do exercício até ele ficar compreensível.

ROADMAP

Mapa de competências • 24 meses

Trilha principal orientada a engenharia de backend.

As fases abaixo seguem uma relação de dependência: código e fundamentos precedem framework; banco precede ORM; confiabilidade precede escala.

01–06	07–12	13–18	19–24
Java + CS	Spring + Dados	Qualidade + Ops	Arquitetura

Fase	Meses	Domínios	Evidência de conclusão
I · Fundação	1–6	Java, OOP, algoritmos, JVM, Git, Linux, rede	Aplicação CLI testada e documentada.
II · Aplicações	7–12	SQL, PostgreSQL, Spring MVC, REST, JPA, Security	API autenticada com banco e validação.
III · Produção	13–18	Testes, arquitetura, Docker, CI/CD, logs, Redis, mensageria	Serviço implantado, observável e testado.
IV · Avançado	19–24	DDD, concorrência, System Design, resiliência, segurança	Projeto de arquitetura com decisões e métricas.

Revisões obrigatórias a cada trimestre

- **Revisão técnica:** prova prática sem tutorial, com testes e explicação oral/escrita.
- **Revisão do portfólio:** atualizar README, exemplos de uso e decisões arquiteturais.
- **Revisão de lacunas:** repetir conteúdos em que a autonomia ficou abaixo do esperado.
- **Revisão de mercado:** olhar vagas Java backend e comparar responsabilidades, sem trocar a stack a cada tendência.

Ordem importante: JDBC + SQL antes de JPA; **Spring Core antes de Spring Boot avançado**; **monólito modular antes de microsserviços**; **métricas antes de otimizações**.

TRILHA ACELERADA · FASES 0–1

12 meses · diagnóstico e base profunda

Usada somente depois de comprovados os conhecimentos básicos. Meta sugerida: 20 h/semana.

Fase 0 · Semanas 1–2

- Aplicar D1, D2 e D3; preencher a rubrica e abrir issues com cada lacuna.
- Revisar *somente* fundamentos com nota abaixo de 3.
- Gerar projeto Boot 4.1.2 + Java 25 pelo Initializr e ler seu `pom.xml` inteiro.

Fase 1 · Meses 1–2 — Java e banco por dentro

Sprint	Aprendizagem	Evidência
1	Records, sealed types, pattern matching no switch.	Estados inválidos modelados explicitamente.
2	JVM: JIT, G1/ZGC, JFR e ferramentas de profiling.	Captura JFR comentada.
3	Virtual threads, scoped values; limites do pool JDBC.	Comparação de latência sob I/O.
4	CTEs, window functions, índices parciais, planos SQL.	Antes/depois com EXPLAIN.
5	MVCC, isolamentos, locks, SKIP LOCKED.	Teste concorrente reproduzível.
6	JDBC puro + HikariCP, transações e rollback.	Transferência com invariantes testadas.

> **Gate F1:** transferência JDBC sem saldo inconsistente em cenário concorrente com 50 tarefas. Registrar duração, tamanho do pool, deadlocks e invariantes por conta. Não confundir 50 tarefas com 50 conexões simultâneas.

Regra de evolução: se o gate falhar, não acelere para Spring; diagnostique transações e locking primeiro.

TRILHA ACELERADA · FASES 2–3

Spring, API, segurança e tenancy

Do IoC até fronteiras efetivas de autorização, com demonstrações executáveis.

Fase 2 · Meses 3–4 — Spring internals

- **Container:** BeanDefinition, BeanFactoryPostProcessor, BeanPostProcessor e ciclo de vida.
- **Proxies:** JDK dinâmico vs subclass proxy; self-invocation, transações e AOP.
- **Auto-configuração:** @AutoConfiguration, @Conditional*, propriedades e ApplicationContextRunner.
- **Spring MVC:** DispatcherServlet, HandlerMapping, argument resolvers, conversores de mensagens.
- **Spring 7:** versionamento de API, retry, limite de concorrência, null-safety JSpecify.
- **Integrações:** RestClient, HTTP Interfaces, timeout, teste com RestTestClient.

> **Gate F2:** starter próprio com ativação condicional, documentação de uso e testes do contexto.

Fase 3 · Meses 5–6 — API profissional

- Contrato OpenAPI antes do controller; ProblemDetail; paginação por cursor.
- Chave de idempotência persistida com restrição única e teste de repetição.
- SecurityFilterChain, JWT como resource server e teste de audience/issuer.
- Keycloak local com usuários e dois tenants; 401, 403 e BOLA/IDOR.
- Multi-tenancy: tenant por coluna ou schema; testar RLS em PostgreSQL.

> **Gate F3:** reservas multi-tenant com prova automatizada de que um usuário de A não pode acessar dados de B, mesmo alterando IDs de recursos na URL.

TRILHA ACELERADA · FASES 4–5

Qualidade, operação e arquitetura

A prioridade é detectar regressões e demonstrar comportamento em falhas, não colecionar ferramentas.

Fase 4 · Meses 7–8

Tema	Experimento verificável
Testes Boot 4	Slices, MockitoBean, Testcontainers com PostgreSQL e ServiceConnection.
Arquitetura	Teste ArchUnit proibindo dependência de adapter → domain inversa.
Mutação	Executar PIT; registrar mutantes sobreviventes e melhorias de teste.
Entrega	Buildpack vs Dockerfile multi-stage; imagem reprodutível.
Observabilidade	Logs, métricas e OTLP com um trace atravessando controller → banco.

Gate F4: deploy reproduzível apenas com README e um runbook para falha induzida.

Fase 5 · Meses 9–10

Tema	Experimento verificável
DDD/Modulith	Agregados, boundaries e verificação de módulos em build.
Eventos	Outbox/registro de publicação; reiniciar e reprocessar sem perder efeito.
Fila	RabbitMQ com DLQ, retry limitado e consumidor idempotente.
Redis	Cache-aside, invalidação e stampede; medir hit rate.
Resiliência	Circuit breaker/bulkhead externos, comparados a retry nativo.
Carga	Rinha de Backend (edital), p95/p99 e comparação de recursos.

Gate F5: pedidos com outbox e teste de mensagem duplicada; ADR explicando por que broker e cache foram introduzidos.

Capstone e preparação profissional

Meses 11–12 dedicados à integração, avaliação externa e narrativa técnica.

Entrega A · System Design

- Estimar RPS, concorrência, volume de dados e orçamento de memória.
- Desenhar fronteiras do domínio e fluxos de falha; registrar três ADRs.
- Medir comportamento sob carga representativa e justificar p95/p99.

Entrega B · SaaS multi-tenant de referência

- Tenants isolados no banco; autenticação pelo Keycloak; autorização por objeto.
- Spring Modulith + publicação de eventos; mensagens e consumidores idempotentes.
- OpenTelemetry, dashboard de erros/latência e runbook reproduzível.
- CI com testes unitários, de integração, segurança e carga.

Entrega C · Portfólio e entrevistas

- Selecionar três repositórios: starter, sistema funcional multi-tenant, benchmark/Rinha.
- README com diagramas, instruções, decisões, testes e números medidos.
- Simular entrevistas e explicar trade-offs sem consultar material.

> Use exemplos e dados sintéticos. Reutilize o domínio de um produto real apenas se o código, os dados e o direito de publicação forem seus.

Distribuição de 20 horas semanais

Estudo	Horas	Produto semanal
Teoria/documentação	4 h	Anotações com conceitos e fontes.
Implementação	10 h	Funcionalidade, spike ou laboratório.
Testes e revisão	3 h	Casos de erro, PR e refatoração.
Algoritmos + inglês técnico	3 h	Exercícios + leitura do release note.

Gate F6: demonstração reproduzível, relatório de segurança, teste de carga, runbook e defesa de arquitetura. Aprovado somente após todos os gates anteriores.

EDIÇÃO REVISADA

Auditoria editorial · o que foi atualizado

Correções técnicas e método de verificação incorporados ao roadmap.

Esta edição foi revisada em 02/11/2026 usando referências primárias. O propósito da revisão é reduzir erros de versão e tornar cada competência verificável por meio de uma entrega.

Ponto	Tratamento na edição revisada
Versões	Java 25 é LTS; Java 27 é versão não-LTS mais recente no período analisado. Use LTS como base; confirme patch no início de cada projeto [R01].
Spring Boot 4	Adotado starter webmvc e catálogo modular de testes; alertas para Jackson 3, Security 7 e migrações [R11, R27, R28].
Persistência	Incluído roteiro de Flyway com módulo PostgreSQL e cenários de rollback e concorrência [R29].
Autorização	Incluídos testes de permissão sobre objetos de outro usuário e uso de Resource Server com JWT validado [R30, R31].
Avaliação	Incluídos critérios de saída por fase, laboratórios de depuração e análise de evidências.

Escopo técnico. Trechos expressamente marcados como esquemáticos precisam de imports, classes e configuração adicionais. As páginas de laboratório diferenciam exemplos didáticos de orientações de produção. Não copie configurações de segurança sem analisar o modelo de autenticação.

Como usar a revisão

- Use as páginas de cronograma para definir o foco da semana, e os módulos para aprofundar o mecanismo.
- Execute os laboratórios somente após estudar seus pré-requisitos; registre erros e explicações.
- Para novas APIs, gere o POM no Spring Initializr, selecione uma versão estável e mantenha o Maven Wrapper no repositório.
- Atualize versões de dependências e imagens após checar notas de segurança e compatibilidade.

Indicador de conclusão: o projeto compila, executa, passa nos testes e pode ser reproduzido por outra pessoa usando as instruções do README.

Trimestre 1 • aprender a programar

Plano mensal, entregas concretas e critérios mínimos.

Mês 01 • Pensamento computacional

PRÁTICA ORIENTADA

Estudar: Problemas, pseudocódigo, tipos, variáveis, entrada/saída, operadores, fluxo condicional e laços.

Entrega: Calculadora Java e avaliador de notas.

Validação: Resolver 15 problemas sem soluções prontas.

Mês 02 • Java essencial

PRÁTICA ORIENTADA

Estudar: Métodos, arrays, strings, escopo, sobrecarga, tratamento inicial de erros, entrada de dados e depuração.

Entrega: Gerenciador de tarefas em Java puro.

Validação: Escrever métodos pequenos e testes de comportamento.

Mês 03 • Java orientado a objetos

PRÁTICA ORIENTADA

Estudar: Classes, objetos, encapsulamento, composição, herança, interfaces, enum, pacotes e coleções.

Entrega: Catálogo de biblioteca em memória.

Validação: Modelar entidades e explicar relações sem usar Spring.

Fechamento do trimestre: reserve a última semana para uma prova sem tutorial: implementar uma mudança pedida, escrever testes, corrigir falhas, explicar suas decisões e registrar o resultado.

Trimestre 2 • dominar a base técnica

Plano mensal, entregas concretas e critérios mínimos.

Mês 04 • Java intermediário

PRÁTICA ORIENTADA

Estudar: Generics, coleções, exceções, Optional, Streams, records, imutabilidade, I/O, conceitos da JVM e garbage collection.

Entrega: Processador de arquivos CSV com relatórios.

Validação: Comparar soluções iterativas e com Streams.

Mês 05 • Algoritmos + ferramentas

PRÁTICA ORIENTADA

Estudar: Big O, busca, ordenação, recursão, hash maps, pilhas/filas, Git, branches, merge, PR e Maven.

Entrega: CLI com testes automatizados e build Maven.

Validação: Resolver problemas, explicar custos e resolver conflitos Git.

Mês 06 • Linux, redes e HTTP

PRÁTICA ORIENTADA

Estudar: Terminal, processos, permissões, variáveis de ambiente, DNS, TCP, TLS, HTTPS, JSON, REST e códigos HTTP.

Entrega: Cliente HTTP Java com logs e tratamento de timeout.

Validação: Explicar o caminho de uma requisição até o servidor.

Fechamento do trimestre: reserve a última semana para uma prova sem tutorial: implementar uma mudança pedida, escrever testes, corrigir falhas, explicar suas decisões e registrar o resultado.

APLICAÇÕES / ANO 1

Trimestre 3 • dados e Spring

Plano mensal, entregas concretas e critérios mínimos.

Mês 07 • SQL e PostgreSQL

PRÁTICA ORIENTADA

Estudar: Tabelas, modelagem, chaves, constraints, joins, índices, GROUP BY, normalização e EXPLAIN.

Entrega: Banco de locadora ou biblioteca com consultas reais.

Validação: Explicar JOIN, índice e plano de execução.

Mês 08 • JDBC, transações e modelagem

PRÁTICA ORIENTADA

Estudar: PreparedStatement, pool de conexões, ACID, isolamento, deadlocks, Flyway e driver específico do banco.

Entrega: CRUD Java JDBC com PostgreSQL e migração SQL.

Validação: Transferência atômica validada em teste.

Mês 09 • Spring Core e Boot

PRÁTICA ORIENTADA

Estudar: IoC, DI, beans, escopos, configuração, starters, autoconfiguração, profiles, Spring MVC e Actuator inicial.

Entrega: Primeira API de tarefas com dados em memória.

Validação: Explicar como um bean é criado e injetado.

Fechamento do trimestre: reserve a última semana para uma prova sem tutorial: implementar uma mudança pedida, escrever testes, corrigir falhas, explicar suas decisões e registrar o resultado.

APLICAÇÕES / ANO 1

Trimestre 4 • APIs de verdade

Plano mensal, entregas concretas e critérios mínimos.

Mês 10 • API REST com Spring MVC

PRÁTICA ORIENTADA

Estudar: Controllers, DTOs, Bean Validation, ProblemDetail, semântica HTTP, paginação, OpenAPI e testes de contrato.

Entrega: API de biblioteca com operações REST completas.

Validação: Retornar erros consistentes e testes HTTP.

Mês 11 • JPA/Hibernate

PRÁTICA ORIENTADA

Estudar: Entities, relacionamentos, lazy/eager, N+1, migrations, repository, JPQL, projeções e @Transactional.

Entrega: API REST conectada ao PostgreSQL com testes de integração.

Validação: Identificar N+1 e justificar fronteiras transacionais.

Mês 12 • Spring Security

PRÁTICA ORIENTADA

Estudar: Sessões e cookies; SecurityFilterChain; autenticação vs autorização; OAuth2/OIDC e JWT validado como resource server.

Entrega: API autenticada por sessão ou IdP externo e com autorização por recurso.

Validação: Demonstrar acesso negado e política least privilege.

Fechamento do trimestre: reserve a última semana para uma prova sem tutorial: implementar uma mudança pedida, escrever testes, corrigir falhas, explicar suas decisões e registrar o resultado.

Trimestre 5 • entregar software confiável

Plano mensal, entregas concretas e critérios mínimos.

Mês 13 • Estratégia de testes

PRÁTICA ORIENTADA

Estudar: JUnit, Mockito, testes de unidade/slice/integração, @AutoConfigureMockMvc no Boot 4, Testcontainers e cobertura por risco.

Entrega: Suite de testes da API com PostgreSQL de teste.

Validação: Reproduzir bugs com testes de regressão.

Mês 14 • Clean Code e design

PRÁTICA ORIENTADA

Estudar: Refactoring, SOLID, composição, coesão, acoplamento, padrões aplicados e organização de módulos.

Entrega: Refatoração documentada do projeto anterior.

Validação: Reduzir complexidade sem alterar comportamento.

Mês 15 • Docker e CI/CD

PRÁTICA ORIENTADA

Estudar: Imagens, Dockerfile, Compose, containers, secrets, build, release, GitHub Actions e deploy.

Entrega: Pipeline automatizado e ambiente de homologação.

Validação: Reconstruir e publicar uma versão do zero.

Fechamento do trimestre: reserve a última semana para uma prova sem tutorial: implementar uma mudança pedida, escrever testes, corrigir falhas, explicar suas decisões e registrar o resultado.

Trimestre 6 • operação e desempenho

Plano mensal, entregas concretas e critérios mínimos.

Mês 16 • Observabilidade

PRÁTICA ORIENTADA

Estudar: Actuator, logs estruturados, métricas, OpenTelemetry, tracing, SLI/SLO, incidentes e backups.

Entrega: Dashboard operacional, alertas básicos e runbook.

Validação: Investigar falha induzida com evidências.

Mês 17 • Cache e performance

PRÁTICA ORIENTADA

Estudar: Profiling, benchmark, Redis, TTL, cache aside, invalidação, índices, pool de conexões e tuning da JVM.

Entrega: Otimização de endpoint lento, antes/depois.

Validação: Comprovar melhoria medindo p95 e consumo.

Mês 18 • Mensageria e assíncrono

PRÁTICA ORIENTADA

Estudar: Filas, eventos, RabbitMQ ou Kafka, retries, DLQ, idempotência, jobs e processamento eventual.

Entrega: Notificações assíncronas em sistema de pedidos.

Validação: Simular duplicidade e falha sem efeito indevido.

Fechamento do trimestre: reserve a última semana para uma prova sem tutorial: implementar uma mudança pedida, escrever testes, corrigir falhas, explicar suas decisões e registrar o resultado.

ARQUITETURA / ANO 2

Trimestre 7 • projetar sistemas

Plano mensal, entregas concretas e critérios mínimos.

Mês 19 • Domínio e modularidade

PRÁTICA ORIENTADA

Estudar: DDD estratégico/tático, agregados, bounded contexts, casos de uso, arquitetura hexagonal e monólito modular.

Entrega: Reorganização de uma aplicação por módulos de negócio.

Validação: Explicar fronteiras e dependências entre módulos.

Mês 20 • JVM e concorrência

PRÁTICA ORIENTADA

Estudar: Heap, stack, GC, threads, virtual threads, Executors, locks, race conditions, profiling e testes concorrentes.

Entrega: Serviço concorrente com métricas e controle de recursos.

Validação: Detectar e eliminar uma condição de corrida.

Mês 21 • System Design

PRÁTICA ORIENTADA

Estudar: Requisitos não funcionais, estimativa de tráfego, throughput, disponibilidade, load balancing e armazenamento.

Entrega: Documento de design de URL shortener ou pedidos.

Validação: Defender escolhas com limites, custos e métricas.

Fechamento do trimestre: reserve a última semana para uma prova sem tutorial: implementar uma mudança pedida, escrever testes, corrigir falhas, explicar suas decisões e registrar o resultado.

Trimestre 8 • consolidar a especialização

Plano mensal, entregas concretas e critérios mínimos.

Mês 22 • Sistemas distribuídos

PRÁTICA ORIENTADA

Estudar: Consistência, particionamento, retries, circuit breaker, outbox, saga conceitual e tolerância a falhas.

Entrega: Fluxo resiliente com outbox e consumidor idempotente.

Validação: Explicar cenários de falha e recuperação.

Mês 23 • Segurança + confiabilidade

PRÁTICA ORIENTADA

Estudar: OWASP Top 10:2025 e API Top 10:2023, BOLA/IDOR, SBOM, rate limits, segredos e proteção de dados.

Entrega: Hardening do projeto e relatório de riscos.

Validação: Demonstrar correções e testes de segurança.

Mês 24 • Capstone e apresentação

PRÁTICA ORIENTADA

Estudar: Arquitetura, implementação, CI/CD, testes, carga, runbook, ADRs, README e entrevista técnica simulada.

Entrega: Sistema final de portfólio com relatório técnico.

Validação: Explicar ponta a ponta e reproduzir a implantação.

Fechamento do trimestre: reserve a última semana para uma prova sem tutorial: implementar uma mudança pedida, escrever testes, corrigir falhas, explicar suas decisões e registrar o resultado.

Lógica de programação em Java

Módulo 01 · objetivos, exercícios, projeto e avaliação.

Módulo 01 Fundamento

Competências a dominar

- Algoritmos; decomposição e invariantes de um problema.
- Tipos primitivos, operadores e conversões; diferenças entre == e igualdade de objetos.
- Fluxo condicional, loops, métodos, arrays, strings e tratamento de entradas.
- Depuração com breakpoints, leitura de stack trace e testes de borda.

Exercícios práticos

- E01: verificar números primos sem biblioteca externa.
- E02: contar frequência de palavras e empates em texto.
- E03: ler transações e imprimir totais por categoria.

Projeto de consolidação

Criar um programa CLI para cadastro, busca, edição e exclusão de itens.

CRITÉRIO DE SAÍDA

Implementar sozinho, sem interface gráfica, com entradas inválidas e testes simples.

Erro comum: Evitar decorar sintaxe sem conseguir formular algoritmo.

Experimento de validação do módulo

- Explique invariantes do algoritmo; construa testes de borda (vazio, repetido, negativo) e determine custo assintótico.
- Entregue código, testes e uma nota curta sobre o que falhou e por quê.
- Nota ≥ 3/4 na rubrica e defesa sem tutorial para avançar.

MÓDULOS FUNDAMENTAIS E AVANÇADOS

Java OOP, APIs e coleções

Módulo 02 · objetivos, exercícios, projeto e avaliação.

Módulo 02 Fundamento

Competências a dominar

- Encapsulamento, composição, interfaces, herança e polimorfismo.
- Imutabilidade, records, enums, generics, equals/hashCode e Comparable/Comparator.
- List, Set, Map, Iterator, Streams, Optional e exceções checked/unchecked.
- Princípios de desenho de APIs, contratos e responsabilidades de classes.

Exercícios práticos

- E01: modelar reserva, quarto e hóspede com invariantes.
- E02: ordenar e agrupar uma lista de pedidos por estado.
- E03: refatorar um switch gigante para estratégias simples.

Projeto de consolidação

Motor de regras de cobrança com múltiplas políticas e testes.

CRITÉRIO DE SAÍDA

Explicar por que usou interface/composição e demonstrar objetos válidos e inválidos.

Erro comum: Não transformar cada classe em herança; preferir composição quando fizer sentido.

Experimento de validação do módulo

- Refatore um domínio com records, sealed e switch exaustivo; compare imutabilidade, encapsulamento e composição.
- Entregue código, testes e uma nota curta sobre o que falhou e por quê.
- Nota ≥ 3/4 na rubrica e defesa sem tutorial para avançar.

APROFUNDAMENTO · MÓDULO 02

Java moderno: tipos como estados

Use recursos estáveis primeiro; identifique explicitamente o que ainda é preview no Java escolhido.

Modelo de domínio orientado a dados

Records modelam dados imutáveis; sealed interfaces expressam um universo fechado de variantes; pattern matching no `switch` permite tratamento exaustivo.

```
sealed interface EstadoConta
    permits Ativa, Bloqueada {}
record Ativa(BigDecimal saldo) implements EstadoConta {}
record Bloqueada(String motivo) implements EstadoConta {}

String resumo(EstadoConta estado) {
    return switch (estado) {
        case Ativa a -> "Saldo: " + a.saldo();
        case Bloqueada b -> "Bloqueada: " + b.motivo();
    };
}
```

Trecho ilustrativo: imports omitidos. A exaustividade decorre do conjunto fechado de subtipos; não transforme todos os objetos em records quando invariantes e identidade exigirem outro desenho.

Outros recursos para dominar

- Text blocks, var (inferência local), Optional, API de Date/Time, collections imutáveis.
- Stream Gatherers e novos recursos de inicialização/construtores: estudar sem antecipar uso indiscriminado.
- Scoped values: contexto de execução imutável e de escopo controlado; cuidado com integrações que dependem de ThreadLocal.
- Structured concurrency: estudar sem pressupor API estabilizada; verificar status na versão de JDK utilizada.

> **Prova M02+:** implemente workflow de contas, adicione novos estados e demonstre que o compilador indica caminhos não tratados.

Estruturas de dados e algoritmos

Módulo 03 · objetivos, exercícios, projeto e avaliação.

Módulo 03 | Fundamento transversal

Competências a dominar

- Big O de tempo e memória; casos médio/pior e trade-offs.
- Arrays, listas, hash tables, pilhas, filas, heap e árvores.
- Busca, ordenação, dois ponteiros, sliding window e recursão.
- Grafos: BFS/DFS básicos; escolha de estrutura conforme acesso e atualização.

Exercícios práticos

- E01: implementar busca binária e descrever seu invariante.
- E02: top K itens mais frequentes com Map + heap.
- E03: detecção de ciclo em grafo dirigido, após estudar DFS.

Projeto de consolidação

Coleção de 30 exercícios com testes e análise de complexidade.

CRITÉRIO DE SAÍDA

Resolver problemas novos e explicar complexidade sem depender de memorização.

Erro comum: Não antecipar problemas muito difíceis antes de dominar padrões básicos.

Experimento de validação do módulo

- Implemente pilha, fila, mapa e busca binária; compare tempos para tamanhos diferentes sem confundir benchmark com Big O.
- Entregue código, testes e uma nota curta sobre o que falhou e por quê.
- Nota ≥ 3/4 na rubrica e defesa sem tutorial para avançar.

JVM, Maven, Git e Linux

Módulo 04 · objetivos, exercícios, projeto e avaliação.

Módulo 04 | Fundamento de plataforma

Competências a dominar

- JDK, JVM, bytecode, compilação, classpath e inicialização de classes.
- Heap, stack, garbage collection, threads e noções de memória.
- Maven: POM, dependências, scopes, plugins, lifecycle e wrapper.
- Git, CLI, processos, variáveis de ambiente, permissões e arquivos.

Exercícios práticos

- E01: construir um JAR executável por Maven.
- E02: simular conflito de merge e resolvê-lo.
- E03: diagnosticar erro de configuração por variável de ambiente.

Projeto de consolidação

Aplicação Java em repositório com pipeline de teste.

CRITÉRIO DE SAÍDA

Reproduzir o build em outra máquina e explicar como a JVM executa o programa.

Erro comum: Não trocar continuamente de build tool; aprenda Maven primeiro.

Experimento de validação do módulo

- Capture um JFR curto, identifique threads e alocações; reproduza o build e resolva conflito real de Git.
- Entregue código, testes e uma nota curta sobre o que falhou e por quê.
- Nota ≥ 3/4 na rubrica e defesa sem tutorial para avançar.

APROFUNDAMENTO · MÓDULO 04

JVM e profiling com evidências

Trate a JVM como plataforma observável, não uma caixa-preta.

Fluxo de investigação

Passo	Pergunta	Evidência
1	É CPU, alocação, lock, I/O ou GC?	Métricas básicas e JFR.
2	Quais stacks e métodos dominam?	JDK Mission Control / flamegraph.
3	Há correlação com carga?	Teste repetido com parâmetros iguais.
4	A alteração resolveu a causa?	Comparação antes/depois com intervalo e dispersão.

Experimentos sugeridos

- Comparar throughput e uso de heap com G1 e ZGC sob a *mesma* carga.
- Produzir pressão de alocação e observar eventos de GC.
- Executar requisições I/O-bound com threads de plataforma e virtual threads; medir limites do pool JDBC.
- Verificar contagem de threads, CPU e conexões ativas antes de concluir que "virtual threads escalam automaticamente".

```
# Captura didática de JFR em processo Java identificado
jcmd <PID> JFR.start name=estudo settings=profile   duration=60s filename=estudo.jfr
jcmd <PID> JFR.check
```

> **Gate:** entregar um relatório de 1 página com hipótese, carga aplicada, JFR, interpretação, mudança única e nova medição.

Redes, HTTP e contratos

Módulo 05 · objetivos, exercícios, projeto e avaliação.

Módulo 05 | Infraestrutura web

Competências a dominar

- DNS, TCP, TLS, cliente-servidor, latência, conexões e timeouts.
- HTTP: métodos, semântica, status, cabeçalhos, cache e cookies.
- REST como estilo arquitetural; recursos, URIs, paginação, filtros e versionamento.
- JSON, idempotência, OpenAPI, validação e códigos de erro.

Exercícios práticos

- E01: capturar uma requisição com curl -v e explicar o resultado.
- E02: desenhar status e payloads de uma API de biblioteca.
- E03: distinguir POST, PUT e PATCH em casos reais.

Projeto de consolidação

Contrato OpenAPI para pedidos e clientes antes de implementar controllers.

CRITÉRIO DE SAÍDA

Justificar cada endpoint, status e erro com semântica HTTP consistente.

Erro comum: Evitar chamar qualquer endpoint JSON de RESTful automaticamente.

Experimento de validação do módulo

- Trace uma requisição HTTPS; explique DNS, status 400/401/403/409/429 e testes de contrato.
- Entregue código, testes e uma nota curta sobre o que falhou e por quê.
- Nota ≥ 3/4 na rubrica e defesa sem tutorial para avançar.

MÓDULOS FUNDAMENTAIS E AVANÇADOS

SQL, PostgreSQL e JDBC

Módulo 06 · objetivos, exercícios, projeto e avaliação.

Módulo 06 | Persistência

Competências a dominar

- Modelo relacional, PK/FK, constraints, normalização e integridade.
- SELECT, JOIN, agregação, subqueries, CTE, views e índices.
- Transações, ACID, isolamentos, concorrência, lock e EXPLAIN ANALYZE.
- JDBC, PreparedStatement, pools, Flyway (starter + módulo do PostgreSQL), rollback.

Exercícios práticos

- E01: escrever 15 queries sobre um esquema de pedidos.
- E02: impedir saldo negativo em transferência transacional.
- E03: medir consulta antes/depois de índice apropriado.

Projeto de consolidação

Aplicação JDBC + PostgreSQL com migrations e transações.

CRITÉRIO DE SAÍDA

Resolver JOINs sem ORM e justificar o limite de uma transação.

Erro comum: Não delegar compreensão de SQL ao Hibernate; não confundir rollback transacional com desfazer migração publicada.

Experimento de validação do módulo

- Rode EXPLAIN (ANALYZE, BUFFERS), demonstre MVCC e compare o comportamento de duas transações concorrentes.
- Entregue código, testes e uma nota curta sobre o que falhou e por quê.
- Nota ≥ 3/4 na rubrica e defesa sem tutorial para avançar.

APROFUNDAMENTO · MÓDULO 06

PostgreSQL: concorrência de verdade

Banco de dados é parte do modelo de consistência da aplicação.

Conceitos que precisam de laboratório

- **MVCC:** snapshots, versões de linha, vacuum e visibilidade de dados.
- **Locks:** SELECT FOR UPDATE, NOWAIT e SKIP LOCKED, incluindo efeitos sob contenção.
- **Índices:** compostos/parciais, seletividade, custos e varreduras de tabela.
- **SQL analítico:** CTEs, window functions e JSONB quando houver requisito real.

Fila simples de jobs no PostgreSQL

```
WITH selecionado AS (
  SELECT id FROM jobs
  WHERE status = 'PENDENTE'
  ORDER BY criado_em, id
  FOR UPDATE SKIP LOCKED
  LIMIT 10
)
UPDATE jobs AS j SET status = 'EM_PROCESSAMENTO'
FROM selecionado AS s
WHERE j.id = s.id
RETURNING j.*;
```

Execute em uma transação curta e complemente com lease, tentativas, heartbeat/recuperação e índice apropriado. SKIP LOCKED é útil para fila de jobs, não uma substituição universal para broker.

Teste concorrente obrigatório

Cenário	Invariante
A→B e B→A simultâneos	Soma de saldos antes = soma depois.
Requisição repetida	Um único efeito por chave de negócio.
Transferência acima do saldo	Nenhuma conta fica inválida.
Falha entre débito e crédito	Rollback integral.

> **Correção importante:** ao adquirir locks de múltiplas contas, ordene os IDs de forma determinística para reduzir deadlocks, e documente política de retry para conflitos transitórios.

Spring Core e Spring Boot

Módulo 07 · objetivos, exercícios, projeto e avaliação.

Módulo 07 Framework

Competências a dominar

- Inversão de controle, beans, injeção por construtor, contextos e ciclo de vida.
- @Configuration, @Bean, @Component, @Service; escopos e proxies.
- Autoconfiguração, starters, configuração externa, profiles e propriedade tipada.
- Spring MVC, DispatcherServlet, aplicação standalone e Actuator introdutório.

Exercícios práticos

- E01: substituir factory manual por DI com interface e implementação.
- E02: usar dois perfis de configuração sem alterar código de negócio.
- E03: diagnosticar dependência circular e propor refatoração.

Projeto de consolidação

API Spring Boot em memória, separando Controller/Service.

CRITÉRIO DE SAÍDA

Descrever criação de beans e fluxo da requisição sem “mágica”.

Erro comum: Evitar injeção de campo e abstrações desnecessárias.

Experimento de validação do módulo

- Mostre ciclo de vida de bean, autoconfiguração condicional, um proxy e um teste com ApplicationContextRunner.
- Entregue código, testes e uma nota curta sobre o que falhou e por quê.
- Nota ≥ 3/4 na rubrica e defesa sem tutorial para avançar.

APROFUNDAMENTO · MÓDULO 07

Spring internals e starter próprio

Entender IoC e AOP para diagnosticar falhas de configuração e transações.

Componentes do container

Componente	O que inspecionar
BeanDefinition	Metadados de criação e escopo do bean.
BeanFactoryPostProcessor	Alterações em definições antes da criação.
BeanPostProcessor	Customização e proxies na inicialização.
AutoConfiguration	Condições dependentes de classes, propriedades e beans.
ApplicationContextRunner	Teste de contexto mínimo com dependências controladas.

Self-invocation e transações

Um método interno do próprio objeto pode chamar outro método anotado com `@Transactional` sem atravessar o proxy. Verifique o mecanismo de interceptação em uso antes de assumir comportamento transacional.

Projeto de biblioteca

- Criar `idempotency-key-spring-boot-starter` para uma interface de armazenamento de chaves.
- Configuração com propriedades tipadas, validade de entradas e ativação condicional.
- Auto-configuração separada do módulo de API, sem dependência circular.
- Testes de contexto: bean presente, bean ausente, override pelo consumidor, propriedade inválida.

> **Gate:** demonstrar o starter em uma aplicação separada e explicar as condições que levam à criação de cada bean.

API REST profissional com Spring MVC

Módulo 08 · objetivos, exercícios, projeto e avaliação.

Módulo 08 | Framework

Competências a dominar

- @RestController, @RequestMapping, @GetMapping, @PostMapping e ResponseEntity.
- DTOs, conversão, @Valid, Bean Validation e erro padronizado.
- Pagination, sorting, filtros, OpenAPI, contratos e compatibilidade de APIs.
- Exceções, @RestControllerAdvice, observabilidade de request e semântica HTTP.

Exercícios práticos

- E01: retornar 201 e localização para criação de recurso.
- E02: testar 400, 404, 409 e 422 conforme o contrato.
- E03: desenhar paginação estável em lista grande.

Projeto de consolidação

API de biblioteca com documentação e teste de contrato.

CRITÉRIO DE SAÍDA

Endpoints consistentes, documentação reproduzível e erros úteis.

Erro comum: Evitar expor entidades JPA diretamente como contrato público.

Experimento de validação do módulo

- Prove versionamento de API, validação e ProblemDetail em testes HTTP; use RestClient em integração externa simulada.
- Entregue código, testes e uma nota curta sobre o que falhou e por quê.
- Nota ≥ 3/4 na rubrica e defesa sem tutorial para avançar.

APROFUNDAMENTO · MÓDULO 08

Spring Framework 7 em APIs

Inclua recursos modernos sem confundir capacidades nativas com bibliotecas externas.

Novidades e prática direcionada

Tema	Experimentação
Versionamento de API	Definir política, resolver versão da requisição e testar compatibilidade/depreciação.
RestClient + HTTP Interfaces	Criar cliente de serviço externo tipado, com timeout, retry e tratamento de erros.
RestTestClient	Testar HTTP em vez de depender apenas de mocks de métodos internos.
Retry nativo	@Retryable com @EnableResilientMethods; só repetir operações seguras/idempotentes.
Limite de concorrência	@ConcurrencyLimit para proteger dependência com capacidade finita.
Null-safety	Ler anotações JSpecify e analisar limites de ferramentas estáticas.

Distinção: retry e concorrência do Spring Framework não substituem circuit breaker, bulkhead, rate limiting e políticas operacionais. Para estes padrões, compare Resilience4j com alternativas.

Contrato mínimo de cliente externo

- Timeout de conexão e resposta configurados explicitamente.
- Erros 4xx e 5xx traduzidos em falhas de domínio adequadas.
- Retries limitados apenas para falhas transitórias e endpoints seguros.
- Teste de dependência lenta, inválida e indisponível com servidor simulado.

Gate: registrar as diferenças observáveis de latência, falha e carga ao aplicar retry e limite de concorrência.

Spring Data JPA e Hibernate

Módulo 09 · objetivos, exercícios, projeto e avaliação.

Módulo 09 | Persistência aplicada

Competências a dominar

- Entidades, @Id, relacionamentos, cascade e estados de persistência.
- JPA repositories, métodos derivados, JPQL, projeções e Specifications.
- Lazy loading, N+1, fetch joins, paginação e índices complementares.
- @Transactional, limites de transação, isolamento, migrations e locking.

Exercícios práticos

- E01: reproduzir e corrigir N+1 com medição de queries.
- E02: implementar transferência entre duas contas em transação.
- E03: comparar DTO projection e carregamento de entidade.

Projeto de consolidação

API PostgreSQL com migrations e testes reais via Testcontainers.

CRITÉRIO DE SAÍDA

Demonstrar rollback, concorrência e consulta eficiente.

Erro comum: Use migrações versionadas. Não use ddl-auto=create em produção nem presuma que @Transactional resolve concorrência.

Experimento de validação do módulo

- Produza um teste que evidencie N+1, controle de transação, lock e falha de persistência com banco real.
- Entregue código, testes e uma nota curta sobre o que falhou e por quê.
- Nota ≥ 3/4 na rubrica e defesa sem tutorial para avançar.

Spring Security e segurança web

Módulo 10 · objetivos, exercícios, projeto e avaliação.

Módulo 10 | Segurança

Competências a dominar

- Autenticação versus autorização; sessões, cookies, CSRF, CORS e TLS.
- Spring Security 7, SecurityFilterChain, filtros, password hashing e segurança de sessão.
- OAuth2/OIDC; validar JWT por assinatura, issuer, audience, exp/nbf e chaves rotativas.
- Permissões por recurso (BOLA/IDOR), OWASP Top 10 web/API, segredos e defesa em profundidade.

Exercícios práticos

- E01: demonstrar 401 vs 403 com políticas explícitas.
- E02: implementar endpoint que só permite acesso ao dono do recurso.
- E03: criar testes de autorização entre usuários distintos.

Projeto de consolidação

API protegida com política de mínimo privilégio e trilha de auditoria.

CRITÉRIO DE SAÍDA

Testar acesso indevido, tokens inválidos e recursos de terceiros.

Erro comum: Nunca desabilitar CSRF ou CORS sem compreender o modelo de autenticação.

Experimento de validação do módulo

- Automatize testes de 401, 403, BOLA entre usuários e isolamento entre tenants.
- Entregue código, testes e uma nota curta sobre o que falhou e por quê.
- Nota ≥ 3/4 na rubrica e defesa sem tutorial para avançar.

Multi-tenancy sem vazamento de dados

Escolher modelo de isolamento explícito e testar a fronteira entre organizações.

Três modelos usuais

Modelo	Vantagem	Risco/Custo
Tenant por coluna	Operação e consultas simples em uma base.	Toda query precisa respeitar tenant; RLS pode ser camada adicional.
Schema por tenant	Isolamento lógico mais nítido.	Migrations por schema, pool e roteamento mais complexos.
Banco por tenant	Maior isolamento operacional.	Custo, provisionamento, backup e manutenção maiores.

Threat model mínimo

- Identidade autenticada carrega organização autorizada; não confie cegamente em header `X-Tenant-Id` enviado pelo cliente.
- Conferir autorização no domínio/serviço e no acesso ao recurso (BOLA), não só no controller.
- Aplicar constraints compostas por tenant quando relevante (tenant_id + identificador).
- Dados de cache, arquivos, filas, exportações e logs também devem carregar contexto autorizado.

Testes indispensáveis

Ação	Resultado
Usuário A lista objetos de A	Somente registros de A.
Usuário A consulta ID válido de B	404 ou 403 conforme contrato; nunca conteúdo de B.
Usuário A altera payload para tenant B	Sem efeito sobre B.
Job assíncrono sem tenant válido	Rejeitado e registrado com segurança.

Gate: suíte de isolamento rodando com PostgreSQL real, incluindo acesso entre tenants após troca de conexões do pool.

LABORATÓRIO DE SEGURANÇA

PostgreSQL Row-Level Security

RLS fornece defesa adicional; sua eficácia depende de roles e de contexto por transação.

Esqueleto didático (ajuste à sua estrutura de roles)

```
ALTER TABLE reservas ENABLE ROW LEVEL SECURITY;

CREATE POLICY reservas_tenant ON reservas
USING (
  tenant_id = current_setting('app.tenant_id', true)::uuid
)
WITH CHECK (
  tenant_id = current_setting('app.tenant_id', true)::uuid
);

BEGIN;
SET LOCAL app.tenant_id = '00000000-0000-0000-0000-000000000001';
SELECT * FROM reservas;
COMMIT;
```

Alertas práticos

- `SET LOCAL` vale na transação atual; evite `SET` persistente que vaze contexto para conexão reutilizada.
- Dono da tabela e roles privilegiadas podem contornar RLS; use usuário de aplicação sem BYPASSRLS.
- Tenha em mente que RLS pode ser bypassado por superuser; verifique `FORCE ROW LEVEL SECURITY` quando aplicável.
- Não confie só na política: faça testes HTTP e SQL de isolamento.
- Migrations e processos batch têm política própria; não abra permissões indiscriminadas em produção.

Teste de aceitação

Grave duas reservas com o mesmo ID lógico em tenants diferentes; alterne o tenant em transações distintas sobre conexões reaproveitadas e demonstre ausência de vazamento.

> **Fonte:** PostgreSQL — Row Security Policies.

Testes automatizados e design de testes

Módulo 11 · objetivos, exercícios, projeto e avaliação.

Módulo 11 Qualidade

Competências a dominar

- JUnit, AssertJ, Mockito, stubs e doubles; limites dos mocks; testes de propriedades em cenários adequados.
- Spring Boot Test, @AutoConfigureMockMvc em testes MockMvc, testes de slices e integração (Boot 4).
- Testcontainers PostgreSQL e @ServiceConnection; fixtures, isolamento e migrações reais.
- Testes de contrato, regressão, carga e riscos de flaky tests.

Exercícios práticos

- E01: escrever teste que reproduz um bug já observado.
- E02: testar status e payload HTTP de erro.
- E03: verificar rollback com banco de teste real.

Projeto de consolidação

Pipeline que executa testes unitários e de integração antes de merge.

CRITÉRIO DE SAÍDA

Proteger fluxos críticos e explicar o que cada nível de teste prova.

Erro comum: Não buscar cobertura de 100% enquanto comportamentos críticos ficam sem teste.

Experimento de validação do módulo

- Faça um teste unitário, um de integração e um de mutação; explique qual regressão cada nível captura.
- Entregue código, testes e uma nota curta sobre o que falhou e por quê.
- Nota ≥ 3/4 na rubrica e defesa sem tutorial para avançar.

Testes além da cobertura

Testes bons reprovam mutações relevantes e protegem decisões arquiteturais.

Ferramentas com objetivos diferentes

Ferramenta	Questão que responde
ArchUnit	O código violou fronteiras e dependências proibidas?
PIT mutation testing	Os testes detectam mudanças perigosas na lógica?
jqwik / property based	A propriedade vale em muitos dados gerados?
Testcontainers	A integração funciona com a tecnologia real?
k6 / Gatling	Como variam latência e taxa de erro sob carga?

Exemplos de propriedades para testar

- Transferência conserva a soma total de saldos quando conclui com sucesso.
- Reservas não podem exceder capacidade disponível.
- Uma mesma Idempotency-Key nunca causa efeito duplicado.
- Usuário não autorizado nunca acessa objeto de outro tenant.

Qualidade mensurável

- Rastreie mutantes sobreviventes relevantes, não apenas um percentual global.
- Crie regras de arquitetura executáveis (ex.: domain não importa infrastructure).
- Registre testes de carga com hardware e dataset descritos no README.

> **Gate:** 1 regressão reproduzida, 1 regra de arquitetura automatizada e 1 teste de propriedade com caso mínimo de falha.

Código limpo e arquitetura modular

Módulo 12 · objetivos, exercícios, projeto e avaliação.

Módulo 12 Design

Competências a dominar

- SOLID como heurísticas de design, alta coesão e baixo acoplamento.
- Refatoração guiada por testes; identificar code smells sem dogmatismo.
- Arquitetura em camadas e hexagonal; portas, adapters e caso de uso.
- Monólito modular, package by feature e evolução sem overengineering.

Exercícios práticos

- E01: separar regra de domínio de transporte HTTP.
- E02: remover acoplamento a banco de um teste unitário.
- E03: escrever ADR justificando uma escolha arquitetural.

Projeto de consolidação

Refatorar API existente mantendo contrato e testes.

CRITÉRIO DE SAÍDA

Demonstrar melhora com alterações localizadas e dependências explícitas.

Erro comum: Evitar criar interfaces para tudo sem substituição ou teste justificável.

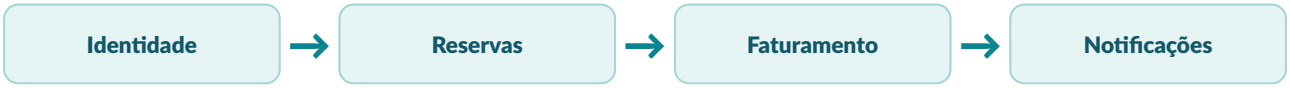
Experimento de validação do módulo

- Verifique fronteiras entre módulos com ArchUnit/Modulith e documente uma decisão arquitetural curta.
- Entregue código, testes e uma nota curta sobre o que falhou e por quê.
- Nota ≥ 3/4 na rubrica e defesa sem tutorial para avançar.

Monólito modular verificável

Spring Modulith é um meio de verificar fronteiras e eventos, não uma obrigação arquitetural.

Proposta de módulos



Exercício completo

- Defina interfaces públicas entre módulos e limite acesso a pacotes internos.
- Execute validação de fronteiras de módulos e crie teste que falha ao importar classes internas.
- Publique evento de `ReservaConfirmada` após a transação de domínio.
- Verifique registro de publicação de eventos; simule falha do consumidor e posterior republicação.
- Documente se há garantia de at-least-once, possibilidade de duplicação e política de idempotência.

Precisão técnica: o registro de publicação do Modulith auxilia a entrega confiável de eventos; não equivale, em todos os cenários, a um outbox completo entre banco e broker sem configuração e verificação do caminho de publicação.

Gate: prova automatizada da fronteira + experimento de falha/recuperação de evento + ADR de monólito modular vs microsserviços.

Docker, CI/CD e entrega

Módulo 13 · objetivos, exercícios, projeto e avaliação.

Módulo 13 | Operações

Competências a dominar

- Dockerfile multi-stage, imagens, containers, redes, volumes e Compose.
- Configuração 12-factor, perfis, segredos e migrações de banco.
- GitHub Actions: build, testes, análises e publicação de artefatos.
- Deploy, health checks, rollback, estratégia de release e backups.

Exercícios práticos

- E01: subir app e banco com Compose em isolamento.
- E02: quebrar pipeline propositalmente e diagnosticar a causa.
- E03: simular deploy ruim e rollback documentado.

Projeto de consolidação

Aplicação reproduzível localmente e implantada em ambiente público seguro.

CRITÉRIO DE SAÍDA

Rodar build/teste/deploy a partir de instruções claras.

Erro comum: Não embutir senha em Dockerfile, logs ou repositório.

Experimento de validação do módulo

- Recrie uma imagem e pipeline a partir de zero; demonstre rollback e análise de dependências vulneráveis.
- Entregue código, testes e uma nota curta sobre o que falhou e por quê.
- Nota ≥ 3/4 na rubrica e defesa sem tutorial para avançar.

ENTREGA · MÓDULO 13

Buildpacks, SBOM e operação

A entrega de software inclui origem verificável de dependências e procedimentos de recuperação.

Pipeline de referência

Estágio	Critério observável
Build	Maven Wrapper, JDK fixado e dependências resolvidas.
Test	JUnit, integração PostgreSQL e análise estática.
Supply chain	SBOM CycloneDX e varredura de dependências com política de triagem.
Imagem	Buildpacks via spring-boot:build-image ou Dockerfile multi-stage.
Entrega	Tags rastreáveis, variáveis externas, secrets fora do repositório.
Operação	Health/probes, requests/limits, logs e runbook.

Tópicos opcionais após baseline funcional

- AOT, AppCDS e imagem nativa GraalVM: comparar startup, heap, tempo de build e limitações.
- Kubernetes básico: liveness, readiness, requests/limits e rollout; testar thresholds com cuidado.
- Não expor endpoints internos do Actuator sem autenticação e controle de rede.

Experimento obrigatório

Produza duas imagens (Buildpack e Dockerfile), meça tamanho, startup, uso de memória e tempo de build em ambiente equivalente. Publique números e procedimentos, não uma preferência genérica.

> **Gate:** reproduzir do zero o deploy e demonstrar recovery rollback após falha simulada.

Observabilidade e desempenho

Módulo 14 · objetivos, exercícios, projeto e avaliação.

Módulo 14 Operações

Competências a dominar

- Logs com correlação e níveis, métricas Micrometer/Actuator e traces.
- Latência p50/p95/p99, throughput, erro, saturação e SLI/SLO.
- Profiling JVM, GC, CPU, memória, pool JDBC, consultas lentas.
- Benchmark controlado, carga, capacidade e interpretação de resultados.

Exercícios práticos

- E01: investigar endpoint lento por logs/tracing.
- E02: coletar p95 antes e após otimização.
- E03: criar runbook para falha do banco de dados.

Projeto de consolidação

Relatório de performance com baseline e melhorias reprodutíveis.

CRITÉRIO DE SAÍDA

Mostrar causa provável a partir de dados, não de palpite.

Erro comum: Não otimizar sem medição e sem verificar regressões.

Experimento de validação do módulo

- Meça p95/p99, erro e saturação; correlacione um trace a logs estruturados e identifique um gargalo real.
- Entregue código, testes e uma nota curta sobre o que falhou e por quê.
- Nota ≥ 3/4 na rubrica e defesa sem tutorial para avançar.

Cache, filas e processamento assíncrono

Módulo 15 · objetivos, exercícios, projeto e avaliação.

Módulo 15 | Escalabilidade

Competências a dominar

- Redis, TTL, cache-aside, invalidação e stampede.
- Mensageria: RabbitMQ ou Kafka, publisher/consumer e ordenação.
- Retry com backoff, DLQ, idempotência e deduplicação.
- Consistência eventual, outbox transacional e tarefas agendadas.

Exercícios práticos

- E01: medir cache hit rate e latência.
- E02: enviar mensagem duplicada sem duplicar cobrança.
- E03: simular falha do consumidor e reprocessamento.

Projeto de consolidação

Pedidos com confirmação assíncrona e outbox ou alternativa justificada.

CRITÉRIO DE SAÍDA

Explicar garantia de entrega e estratégia de reprocessamento.

Erro comum: Não iniciar com Kafka ou microsserviços sem necessidade operacional.

Experimento de validação do módulo

- Teste uma mensagem duplicada, a indisponibilidade do broker e a invalidação do cache.
- Entregue código, testes e uma nota curta sobre o que falhou e por quê.
- Nota ≥ 3/4 na rubrica e defesa sem tutorial para avançar.

DDD, JVM e concorrência avançada

Módulo 16 · objetivos, exercícios, projeto e avaliação.

Módulo 16 | Arquitetura

Competências a dominar

- Modelagem de domínio: entidades, value objects, agregados e invariantes.
- Bounded contexts, linguagem ubíqua, eventos de domínio e módulos.
- Threads, virtual threads, Executors, locks, concorrência e race conditions.
- Heap, GC, instrumentação e trade-offs entre recursos e latência.

Exercícios práticos

- E01: modelar limites do domínio em um evento de pedido.
- E02: provar uma race condition e corrigir com controle adequado.
- E03: comparar concorrência bloqueante e não bloqueante em teste.

Projeto de consolidação

Aplicação modular com caso de uso de negócio complexo e teste concorrente.

CRITÉRIO DE SAÍDA

Justificar fronteiras de módulos e manter invariantes do domínio.

Erro comum: Evitar tratar virtual threads como solução automática para todos os gargalos.

Experimento de validação do módulo

- Compare virtual threads e pool de conexões; execute JFR e provoque uma condição de corrida reproduzível.
- Entregue código, testes e uma nota curta sobre o que falhou e por quê.
- Nota ≥ 3/4 na rubrica e defesa sem tutorial para avançar.

Virtual threads e estrutura de tarefas

Separar simultaneidade, paralelismo, limite de recursos e segurança da memória.

Conceitos fundamentais

- Virtual threads ajudam especialmente cargas com I/O bloqueante; não aceleram trabalho CPU-bound automaticamente.
- O limite de conexões do PostgreSQL/HikariCP continua existindo; milhares de threads não implicam milhares de transações.
- Race conditions, visibilidade e ordenação exigem raciocínio sobre modelo de memória e invariantes.
- Scoped values são diferentes de ThreadLocal mutável; avaliar propagação de contexto em bibliotecas.
- Structured concurrency em Java 25 é recurso preview; requer habilitação e cuidados ao evoluir a aplicação.

Experimento comparativo

Cenário	Medir
Pool de 10 conexões e 500 tarefas	Espera por conexão, p95 e saturação.
Cliente HTTP com latência de 250 ms	Throughput e threads ativas.
Cálculo CPU-bound	Uso de CPU e ganho real, se houver.
Cancelamento de requisições	Recursos liberados e tarefas órfãs.

> Nunca apresente resultados de máquina local como benchmark universal; anote CPU, RAM, JDK, tamanho do pool, dataset e modo de execução.

> **Gate:** explicar por que aumentar a contagem de threads não resolve saturação do banco e propor limite de concorrência verificável.

System Design e sistemas distribuídos

Módulo 17 · objetivos, exercícios, projeto e avaliação.

Módulo 17 Avançado

Competências a dominar

- Estimativas: RPS, armazenamento, concorrência, rede e custo.
- Disponibilidade, redundância, balanceamento, cache, particionamento e replicação.
- Consistência, CAP, idempotência, timeouts, circuit breakers e bulkheads.
- Sagas, outbox, observabilidade, segurança e análise de falhas.

Exercícios práticos

- E01: estimar carga de 100 mil usuários ativos (hipóteses explícitas).
- E02: desenhar um fluxo de pedidos resistente a retries.
- E03: comparar monólito modular versus microsserviços em ADR.

Projeto de consolidação

Documento de arquitetura + protótipo funcional + relatório de testes de carga.

CRITÉRIO DE SAÍDA

Defender decisões em cenários de erro e escala com números plausíveis.

Erro comum: Não confundir complexidade distribuída com qualidade técnica.

Experimento de validação do módulo

- Apresente ADR com requisitos e métricas; simule falha e defenda por que não migrar cedo para microsserviços.
- Entregue código, testes e uma nota curta sobre o que falhou e por quê.
- Nota ≥ 3/4 na rubrica e defesa sem tutorial para avançar.

PLANO DE ARRANQUE

Primeiras 8 semanas • 1–4

Roteiro de sessões que transforma o roadmap em rotina.

Objetivo das semanas iniciais: escrever Java todos os dias de estudo, construir programas pequenos e ganhar autonomia antes de Spring.

Semana 01 • Pensamento computacional

Roteiro: Seg: instalar JDK/IDE e imprimir resultados. Ter: variáveis e tipos. Qua: operadores e conversões. Qui: condicionais. Sex: 5 problemas de entrada/saída. Fim de semana: calculadora.

Entrega: Calculadora CLI com validação de divisão por zero e README.

Semana 02 • Laços e funções

Roteiro: Seg: while/for. Ter: funções e parâmetros. Qua: escopo e retorno. Qui: depurador e stack trace. Sex: 5 exercícios de repetição. Fim de semana: jogo de adivinhação.

Entrega: Jogo com tentativas, mensagens de erro e testes simples.

Semana 03 • Arrays, strings e objetos

Roteiro: Seg: arrays. Ter: strings. Qua: Arrays/ArrayList. Qui: classes simples. Sex: buscar itens. Fim de semana: catálogo de tarefas.

Entrega: Tarefas com operações criar, listar, concluir e remover.

Semana 04 • Primeira revisão prática

Roteiro: Seg: tratamento de exceções. Ter: métodos pequenos. Qua: Git. Qui: testes. Sex: revisão. Fim de semana: reconstruir CLI do zero sem tutorial.

Entrega: Projeto publicado com commits, README e autoavaliação.

Avaliação: realize um exercício inédito de 45–90 minutos, crie testes para entradas inválidas e explique suas funções sem consultar código anterior.

PLANO DE ARRANQUE

Primeiras 8 semanas • 5–8

Roteiro de sessões que transforma o roadmap em rotina.

Objetivo das semanas iniciais: escrever Java todos os dias de estudo, construir programas pequenos e ganhar autonomia antes de Spring.

Semana 05 • OOP na prática

Roteiro: Seg: construtores. Ter: encapsulamento. Qua: composição. Qui: interfaces. Sex: polimorfismo. Fim de semana: motor de descontos.

Entrega: Políticas de desconto sem condicionais gigantes.

Semana 06 • Coleções e generics

Roteiro: Seg: List. Ter: Set. Qua: Map. Qui: equals/hashCode. Sex: generics. Fim de semana: ranking de vendas.

Entrega: Relatório com agrupamentos e ordenação reproduzíveis.

Semana 07 • Arquivos e exceções

Roteiro: Seg: File/Path. Ter: leitura CSV. Qua: IOException. Qui: records. Sex: testes. Fim de semana: consolidação de faturas.

Entrega: Programa lê CSV, valida campos e gera resumo.

Semana 08 • Git, Maven e qualidade

Roteiro: Seg: Maven. Ter: Git branches. Qua: Pull Request. Qui: testes. Sex: debug. Fim de semana: prova de autonomia com mudança surpresa.

Entrega: JAR, testes automatizados, README e PR de revisão.

Avaliação: realize um exercício inédito de 45–90 minutos, crie testes para entradas inválidas e explique suas funções sem consultar código anterior.

PORTFÓLIO ORIENTADO A PROBLEMAS

Projeto 01 • Gestor financeiro CLI

Projetos crescentes para transformar conhecimento em experiência comprovável.

Básico • Java puro • 2–4 semanas

Criar um terminal financeiro para registrar entradas, saídas, categorias e resumos por período. O sistema deve funcionar sem framework web.

Requisitos mínimos

- Criar e editar lançamentos; excluir apenas com confirmação.
- Validar valores positivos, datas e categorias.
- Gerar saldo, total por categoria e resumo mensal.
- Persistir em arquivo CSV/JSON ou formato simples; tratar corrupção de entrada.

Modelo inicial

`Transacao`, `Categoria`, `RelatorioMensal`

Definição de pronto (DoD)

- 15 testes de lógica e validação
- README com comandos para executar
- Exemplo de arquivo de entrada e saída

> **Desafio extra:** Adicionar exportação CSV e orçamento por categoria.

Como apresentar no portfólio

- Descreva o problema e as regras de negócio com exemplos.
- Inclua instruções de execução, arquitetura, testes e API.
- Apresente erros que ocorreram, alternativas consideradas e trade-offs.
- Inclua capturas de métricas ou logs apenas quando comprovarem algo relevante.

Projeto 02 • API de biblioteca

Projetos crescentes para transformar conhecimento em experiência comprovável.

Intermediário • Spring MVC + PostgreSQL • 4–6 semanas

Construir uma API para obras, exemplares, usuários e empréstimos com regras de disponibilidade e prazos.

Requisitos mínimos

- CRUD de livros com paginação e filtros.
- Empréstimo permitido apenas para exemplar disponível.
- Devolução atualiza estado em transação.
- Contrato OpenAPI, erros de domínio e validação.

Modelo inicial

`Livro`, `Exemplar`, `Usuario`, `Emprestimo`

Definição de pronto (DoD)

- Testes unitários dos casos de uso
- Testes de integração com PostgreSQL real
- Migrations e documentação dos endpoints

> **Desafio extra:** Implementar reserva em fila de espera com prazo de expiração.

Como apresentar no portfólio

- Descreva o problema e as regras de negócio com exemplos.
- Inclua instruções de execução, arquitetura, testes e API.
- Apresente erros que ocorreram, alternativas consideradas e trade-offs.
- Inclua capturas de métricas ou logs apenas quando comprovarem algo relevante.

Projeto 03 • Reservas com autenticação

Projetos crescentes para transformar conhecimento em experiência comprovável.

Intermediário • Security + Testes • 4–6 semanas

API de reservas de salas, equipamentos ou horários, com conflitos, perfis e políticas por usuário.

Requisitos mínimos

- Usuário só gerencia suas próprias reservas.
- Não permitir reservas em intervalos sobrepostos.
- Retornar 401/403/409 segundo contrato definido.
- Auditar criação, alteração e cancelamento.

Modelo inicial

`Usuario`, `RecursoReservavel`, `Reserva`, `Auditoria`

Definição de pronto (DoD)

- Testes de autorização entre contas
- Teste de conflito concorrente
- Contrato de erro e fluxos de autenticação

> **Desafio extra:** Adicionar política de cancelamento e eventos de domínio.

Como apresentar no portfólio

- Descreva o problema e as regras de negócio com exemplos.
- Inclua instruções de execução, arquitetura, testes e API.
- Apresente erros que ocorreram, alternativas consideradas e trade-offs.
- Inclua capturas de métricas ou logs apenas quando comprovarem algo relevante.

Projeto 04 • Plataforma de pedidos

Projetos crescentes para transformar conhecimento em experiência comprovável.

Avançado • Mensageria + Redis • 6–9 semanas

Implementar catálogo, carrinho, compra e confirmação assíncrona com consistência explícita.

Requisitos mínimos

- Preço e estoque validados no ato da compra.
- Transação protege criação do pedido e estado inicial.
- Envio de evento por outbox ou mecanismo justificado.
- Consumidor tolera retries e mensagens duplicadas.
- Logs de correlação e métricas de fluxo.

Modelo inicial

`Produto`, `Estoque`, `Pedido`, `ItemPedido`, `EventoOutbox`

Definição de pronto (DoD)

- Teste de idempotência e falha
- Benchmark e relatório de latência
- Runbook de reprocessamento e alertas

> **Desafio extra:** Adicionar rate limit, DLQ e estratégia de cache invalidation.

Como apresentar no portfólio

- Descreva o problema e as regras de negócio com exemplos.
- Inclua instruções de execução, arquitetura, testes e API.
- Apresente erros que ocorreram, alternativas consideradas e trade-offs.
- Inclua capturas de métricas ou logs apenas quando comprovarem algo relevante.

PORTFÓLIO ORIENTADO A PROBLEMAS

Projeto 05 • Capstone de arquitetura

Projetos crescentes para transformar conhecimento em experiência comprovável.

Avançado • Sistema de portfólio • 8–12 semanas

Escolha um problema real: marketplace de serviços, gestão de entregas ou SaaS de assinaturas. Trate-o como um produto com requisitos e limites documentados.

Requisitos mínimos

- Escrever requisitos, NFRs, modelo de domínio e ADRs.
- Implementar monólito modular como baseline e justificar componentes externos.
- Criar CI/CD, segurança, testes, métricas e backups.
- Fazer testes de carga com hipóteses e comparar resultados.
- Preparar demo técnica e documentação de incidentes.

Modelo inicial

`Módulos de domínio separados`, `Banco e migrações`, `Filas/cache apenas se justificados`

Definição de pronto (DoD)

- Repositório público sem segredos
- Diagramas e decisões arquiteturais
- Métricas de desempenho e testes reproduzíveis

> **Desafio extra:** Avaliar se a extração de um serviço reduz ou aumenta a complexidade total.

Como apresentar no portfólio

- Descreva o problema e as regras de negócio com exemplos.
- Inclua instruções de execução, arquitetura, testes e API.
- Apresente erros que ocorreram, alternativas consideradas e trade-offs.
- Inclua capturas de métricas ou logs apenas quando comprovarem algo relevante.

PORTFÓLIO AVANÇADO

Quatro projetos complementares

Os cinco projetos originais permanecem; estes demonstram internals, concorrência e decisões com números.

A • Starter próprio de idempotência

Entregas: autoconfiguração condicional, propriedades, implementação de store substituível, testes de ApplicationContextRunner, README e aplicativo consumidor.

B • Rinha de Backend 2024-Q1

Seguir o edital histórico oficial, medir carga real e p95/p99, publicar limites do hardware e um ADR. Não alegar comparabilidade fora do mesmo cenário de teste.

C • Fila de jobs: PostgreSQL x RabbitMQ

Mesma carga e semântica para ambas as abordagens; comparar throughput, latência, custo de operação, recuperação e duplicidade.

D • SaaS B2B multi-tenant

Reservas/pedidos com tenant isolado, RLS, OAuth2/Keycloak, Modulith, publicação de eventos, OTel, CI/CD e runbook.

Regra de portfólio: apresente evidências: scripts de teste, logs, números medidos, ADRs, README e falhas conhecidas; não só screenshots. Use dados sintéticos e código publicável.

PROJETO COMPLEMENTAR A

Starter Spring Boot: contrato e testes

Demonstra autoconfiguração e diagnóstico de condições, não apenas uso de annotations.

Contrato funcional

- `idempotency.enabled=true|false` controla presença do interceptor/serviço.
- Consumers podem fornecer bean próprio de armazenamento, sem conflito.
- Para mesma chave + mesmo payload, retornar resultado previamente persistido; chave com payload incompatível deve gerar erro contratual.
- Definir TTL, tamanho e proteção contra condição de corrida; documentar efeito de falhas entre gravações.

Casos de teste de referência

ID	Dado	Esperado
A01	Feature desativada	Bean não registrado.
A02	Feature ativa sem store custom	Store default registrado.
A03	Bean custom presente	Default não sobrescreve.
A04	Mesmo key + payload	Um efeito, mesma resposta contratada.
A05	Mesmo key + payload diferente	409 Conflict ou contrato explícito equivalente.

Entrega verificável

Separar `autoconfigure`, `starter` e `demo`; incluir diagrama de criação de beans, changelog e instruções para habilitar/desabilitar.

> **Aprovação:** casos A01–A05 automatizados e API usada com sucesso em projeto consumidor independente.

PROJETO COMPLEMENTAR C

Fila PostgreSQL × RabbitMQ

Uma comparação experimental honesta entre duas soluções para o mesmo requisito.

Carga e contrato comuns

Gerar 10 mil jobs sintéticos, payload uniforme, 1–4 consumidores e dois perfis de falha (consumer crash e indisponibilidade da dependência). Mesma máquina, versão Java e política de retries.

Métricas e critérios

Métrica	Por que registrar
Throughput (jobs/s)	Produtividade sob carga definida.
p50 / p95 / p99	Cauda de latência e experiência de fila.
Falhas e duplicações	Garantia real de processamento idempotente.
Tempo de recuperação	Comportamento após falha/retorno.
Complexidade operacional	Processos, configuração, backup e observabilidade.

Cuidado com semântica

- Comparar at-least-once com at-least-once; se houver exactly-once aparente, explicitar mecanismo de deduplicação.
- No PostgreSQL, tratar leases, requeue e retenção; no RabbitMQ, ack/nack, prefetch, DLQ e retries.
- Repetir múltiplas rodadas, descartar warmup e publicar resultados, scripts e limitações.

> **Aprovação:** gráfico comparável, tabela de resultados, ADR de escolha por cenário e suíte de testes de falha.

PROJETO COMPLEMENTAR D

Capstone SaaS B2B multi-tenant

Projeto final para demonstrar segurança, domínio, consistência e manutenção operacional.

Backlog obrigatório

Epic	Aceite mínimo
Identidade	OIDC/Keycloak e papéis de tenant separados.
Dados	RLS verificada e políticas de backup/migração.
Domínio	Reserva não duplica; pedido conserva totais; invariantes testadas.
Eventos	Outbox/registro com reprocessamento e consumidor idempotente.
Resiliência	Timeout, retry limitado, circuit breaker quando justificável.
Observabilidade	Logs estruturados, métricas p95 e trace de ponta a ponta.
Entrega	CI, imagem, migrations, scripts de seed e runbook.

Critérios públicos de avaliação

- A implementação sobe via instruções no README sem variáveis secretas no repositório.
- Nenhuma tentativa de acesso entre tenants retorna dados alheios.
- Falhas de dependência externa não causam duplicidade do efeito.
- A demonstração inclui p95/p99 e taxa de erros sob carga documentada.
- Há decisões arquiteturais (ADRs) e limitações conhecidas.

> O projeto não deve reutilizar códigos ou bases de clientes sem titularidade/autorização para publicação.

EXERCÍCIOS PARA PRÁTICA DELIBERADA

Banco de exercícios • Fundamentos

Problemas graduados, com critérios claros de correção.

#	Desafio	Entrega mínima
01	Contador de frequência	Receba uma frase, normalize letras e retorne palavras e contagens.
02	Validação de CPF fictício	Crie uma rotina de validação de formato para IDs fictícios; não use dados reais.
03	Paginação em memória	Retorne uma fatia ordenada de uma lista, cobrindo página vazia.
04	Parser CSV	Leia registros, valide colunas e gere relatório de linhas inválidas.
05	Busca binária	Implemente e teste listas vazias, duplicatas e valores ausentes.
06	Ranking	Retorne os 5 produtos mais vendidos e justifique a complexidade.
07	Contagem regressiva	Agende estados em memória sem usar Thread.sleep em lógica de domínio.
08	Conflitos de horários	Detecte sobreposição de intervalos com limites inclusivos/exclusivos.
09	LRU conceitual	Implemente uma política LRU simplificada e explique uso de memória.
10	Merge de dados	Consolide duas listas com IDs repetidos sem perder registros.

Regra: registre hipótese, solução, teste, complexidade ou risco e uma melhoria possível. O objetivo não é concluir todos no mesmo mês.

EXERCÍCIOS PARA PRÁTICA DELIBERADA

Banco de exercícios • Backend e arquitetura

Problemas graduados, com critérios claros de correção.

#	Desafio	Entrega mínima
11	Semântica HTTP	Desenhe resposta 201, 400, 404 e 409 para uma API real.
12	Transação bancária	Garanta transferência atômica; teste rollback em falha.
13	N+1	Identifique consultas extras em relacionamento JPA e meça uma correção.
14	Autorização horizontal	Impeça acesso ao pedido de outro usuário autenticado.
15	Idempotência	Processe duas requisições com mesma chave sem duplicar efeito.
16	Cache	Defina TTL e política de invalidação para catálogo mutável.
17	Retry	Simule timeout; implemente retry limitado com backoff e log.
18	Testcontainers	Teste persistência real com base isolada e migrations.
19	Carga	Meça p95 e taxa de erros em três níveis de concorrência.
20	System Design	Projete serviço de notificações e identifique pontos únicos de falha.

Regra: registre hipótese, solução, teste, complexidade ou risco e uma melhoria possível. O objetivo não é concluir todos no mesmo mês.

LABORATÓRIO DE CÓDIGO

Laboratório • Java bem modelado

Trechos didáticos: complete imports, contexto e testes ao implementar.

Exemplo de regra de domínio sem Spring. Comece por aqui para entender objetos válidos, invariantes e exceções.

```
import java.math.BigDecimal;
import java.util.Objects;

public final class Conta {
    private final String numero;
    private BigDecimal saldo;

    public Conta(String numero, BigDecimal saldoInicial) {
        this.numero = Objects.requireNonNull(numero);
        if (saldoInicial == null || saldoInicial.signum() < 0) {
            throw new IllegalArgumentException("Saldo inválido");
        }
        this.saldo = saldoInicial;
    }

    public void debitar(BigDecimal valor) {
        if (valor == null || valor.signum() <= 0) {
            throw new IllegalArgumentException("Valor inválido");
        }
        if (saldo.compareTo(valor) < 0) {
            throw new IllegalStateException("Saldo insuficiente");
        }
        saldo = saldo.subtract(valor);
    }

    public BigDecimal saldo() { return saldo; }
}
```

O que investigar

- Por que `BigDecimal` é mais apropriado que `double` para valores monetários?
- Como o construtor impede um objeto inválido?
- Quais testes cobririam zero, negativo, saldo insuficiente e valor exato?
- Quais informações adicionais seriam necessárias numa transferência real entre contas?

> **Exercício de extensão:** implemente uma classe de serviço que transfira valores entre duas contas, e depois discuta por que persistência real exige transação no banco.

LABORATÓRIO DE CÓDIGO

Laboratório • Controller REST

Um endpoint pequeno e com responsabilidade explícita.

Em Spring, o controller recebe e valida entradas HTTP; o serviço concentra o caso de uso. Não coloque a regra inteira dentro do endpoint.

```
import jakarta.validation.Valid;
import jakarta.validation.constraints.NotBlank;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.UUID;

record CriarTarefaRequest(@NotBlank String titulo) {}
record TarefaResponse(UUID id, String titulo) {}

@RestController
@RequestMapping("/api/v1/tarefas")
class TarefaController {
    private final TarefaService service;

    TarefaController(TarefaService service) {
        this.service = service;
    }

    @PostMapping
    ResponseEntity<TarefaResponse> criar(
            @Valid @RequestBody CriarTarefaRequest req) {
        TarefaResponse criada = service.criar(req);
        return ResponseEntity.status(HttpStatus.CREATED)
                             .body(criada);
    }
}
```

Trecho didático: a interface/classe `TarefaService` precisa ser implementada; o projeto necessita das dependências apropriadas de Web MVC e Validation.

Experimentos obrigatórios

- Enviar JSON válido: verificar status `201` e corpo.
- Enviar título vazio: verificar validação, sem persistência.
- Definir contrato de resposta para recurso ausente e conflito.
- Criar um teste de integração HTTP para validar o contrato.

Versão Boot 4: use `spring-boot-starter-webmvc` e, para testes MVC, considere `spring-boot-starter-webmvc-test`. `spring-boot-starter-web` está depreciado na série 4. [R27]

LABORATÓRIO DE CÓDIGO

Laboratório · Transações e persistência

A anotação não elimina os problemas de concorrência.

A fronteira transacional costuma coincidir com uma unidade de trabalho de negócio. Nesta página, examine o que deve acontecer quando algo falha entre duas escritas.

```
@Service
class TransferenciaService {
    private final ContaRepository contas;
    TransferenciaService(ContaRepository contas) {
        this.contas = contas;
    }

    @Transactional
    public void transferir(long origemId, long destinoId,
                           BigDecimal valor) {
        if (origemId == destinoId) throw new IllegalArgumentException();
        long menor = Math.min(origemId, destinoId);
        Conta a = contas.lockById(menor).orElseThrow();
        Conta b = contas.lockById(Math.max(origemId, destinoId))
            .orElseThrow();
        Conta origem = origemId == menor ? a : b;
        Conta destino = origemId == menor ? b : a;
        origem.debitar(valor);
        destino.creditar(valor);
    }
}
```

Código esquemático: lockById deve consultar com @Lock(PESSIMISTIC_WRITE); adquira IDs em ordem global e trate deadlocks transitórios. suponha entidades JPA gerenciadas, repositório implementado, validações de domínio e imports. Não é receita pronta para transferência financeira em produção.

O que ainda precisa ser resolvido?

- Concorrência entre transferências, isolamento e travamento de registros.
- Validação de origem diferente do destino e valores corretos.
- Regras de rollback e exceções; cuidado com chamadas internas que não atravessam proxy.
- Idempotência se a transferência for acionada por uma requisição repetida.
- Testes reais com PostgreSQL e execução concorrente.

> **Prova do módulo:** construa um teste que provoque falha após o débito e comprove que a transação não ficou parcialmente gravada.

LABORATÓRIO DE CÓDIGO

Laboratório • Segurança e testes

Segurança orientada a comportamento verificável.

Autorização explícita no Spring Security

```
@Bean
SecurityFilterChain apiSecurity(HttpSecurity http)
        throws Exception {
    http.authorizeHttpRequests(auth -> auth
        .requestMatchers("/actuator/health").permitAll()
        .anyRequest().authenticated());
    return http.build();
}
```

Trecho ilustrativo de autorização. Imports e mecanismo de autenticação devem ser configurados conforme sua aplicação. Não desligue CSRF por reflexo.

Teste de domínio com JUnit

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
import java.math.BigDecimal;

class ContaTest {
    @Test
    void naoPermiteDebitarAcimaDoSaldo() {
        var conta = new Conta("001", new BigDecimal("30.00"));
        assertThrows(IllegalStateException.class,
            () -> conta.debitar(new BigDecimal("31.00")));
        assertEquals(new BigDecimal("30.00"), conta.saldo());
    }
}
```

Checklist de verificação

- Teste usuário anônimo, autenticado e autenticado sem permissão.
- Teste recurso de outro usuário (autorização horizontal).
- Não exponha senhas, tokens ou dados sensíveis em logs.
- Documente configuração de sessão/cookie ou JWT e o motivo.
- Execute testes também com banco e fluxo HTTP completos.

ERRATA DOS LABORATÓRIOS

Dinheiro, locking e Testcontainers

Três ajustes de precisão que melhoram a correção dos exemplos já presentes.

01 • Comparação de BigDecimal

```
// equals() compara também a escala:
new BigDecimal("30.0").equals(new BigDecimal("30.00"));
// false

// Igualdade numérica para este teste:
assertThat(saldo).isEqualByComparingTo("30.00");
```

02 • Aquisição de locks determinística

Em transferências cruzadas, adquirir conta A e depois B em ordem variável pode causar deadlock. Ordene IDs antes de executar consultas `SELECT ... FOR UPDATE`, mantendo uma transação única.

```
long primeiro = Math.min(origemId, destinoId);
long segundo  = Math.max(origemId, destinoId);
// lock(primeiro); lock(segundo); depois identifica origem/destino
// Debitar/creditar dentro da mesma transação.
```

03 • Testcontainers 2 e dependências

Artefatos e pacotes podem mudar entre gerações. Gere as dependências por Initializr e confirme coordenadas no catálogo compatível com o BOM do projeto; um exemplo de referência é `testcontainers-postgresql`.

> **Gate do laboratório:** rodar teste de saldos com escalas distintas, transferência cruzada concorrente e integração PostgreSQL real sem dependências inconsistentes.

LABORATÓRIO ATUALIZADO

Spring Boot 4 · compatibilidade e starters

Convenções e dependências verificadas nas referências do Spring Boot 4.

O ecossistema Spring evolui por versões coordenadas. Use o BOM da versão Spring Boot escolhida em vez de informar manualmente versões para todas as dependências gerenciadas [R27].

Necessidade	Dependência / abordagem
API HTTP MVC	spring-boot-starter-webmvc
DTO e Bean Validation	spring-boot-starter-validation
Persistência JPA	spring-boot-starter-data-jpa
Migração de banco	spring-boot-starter-flyway + org.flywaydb:flyway-database-postgresql
Segurança básica	spring-boot-starter-security
API Resource Server JWT	spring-boot-starter-security-oauth2-resource-server
Testes gerais	spring-boot-starter-test
Testes MVC e segurança	spring-boot-starter-webmvc-test e spring-boot-starter-security-test
Teste com PostgreSQL real	Testcontainers + spring-boot-testcontainers
Operação e monitoramento	spring-boot-starter-actuator

Compatibilidade e migração

- Para a linha 4.0, a documentação registra suporte de Java 17 até Java 26; Java 25 LTS é recomendação de aprendizagem, não requisito mínimo do framework [R04].
- Spring Boot 4 favorece Jackson 3 (módulos `tools.jackson`). Ao copiar código da série 3, procure imports e customizações antigos [R11, R28].
- Spring Security 7 e a infraestrutura de testes sofreram mudanças. Consulte os guias de migração em vez de misturar dependências Spring 3/4.
- Em uma aplicação real, aplique atualizações de patch e verifique CVEs; não congele uma versão específica por 24 meses.

Exercício: gere um projeto somente com Web MVC, Validation e Test. Inspecione `mvnw dependency:tree` e descubra qual dependência fornece cada recurso. Repita após adicionar JPA e Security.

LABORATÓRIO EXECUTÁVEL

Primeira API Spring • execução reproduzível

Projeto mínimo que demonstra Spring MVC; o build é criado pelo Initializr.

Este laboratório produz uma resposta HTTP sem exigir banco, autenticação ou containers. Realize após concluir os módulos de Java, HTTP e Maven.

1 • Gere o projeto

- Abra Spring Initializr: Maven, Java, Java 25 e linha estável do Boot 4; não escolha milestone/snapshot.
- Dependências iniciais: Spring Web MVC, Validation e Spring Boot Test. Use o Maven Wrapper gerado.
- Coloque as classes a seguir sob o mesmo pacote raiz da aplicação principal.

2 • Crie um endpoint mínimo

```
package br.com.estudos.api;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
class SaudacaoController {
    @GetMapping("/api/saudacao")
    Mensagem responder() {
        return new Mensagem("Olá, backend!");
    }
}

record Mensagem(String texto) {}
```

3 • Execute e verifique

```
# PowerShell - na pasta do projeto
.\mvnw.cmd test
.\mvnw.cmd spring-boot:run

# Em outra janela
curl.exe -i http://localhost:8080/api/saudacao
```

Resultado esperado: status `200 OK` e JSON com a propriedade `texto`. Se a porta 8080 estiver ocupada, configure outra porta e repita.

> **Critério de aprovação:** explicar o papel do controller, da anotação, do record, da serialização JSON e do Maven Wrapper; alterar o endpoint e registrar um teste de regressão.

LABORATÓRIO PRÁTICO

Contratos HTTP • validação e falhas

Casos de sucesso e falha devem ser tão bem definidos quanto o fluxo principal.

Uma API de qualidade possui contratos previsíveis tanto para caminhos de sucesso quanto para erros. Estude status HTTP, cabeçalhos e o formato Problem Details (RFC 9457).

Cenário	Resposta esperada	Teste sugerido
Recurso criado	201 Created; Location quando aplicável	Verificar status, corpo e identificador.
Entrada inválida	400 Bad Request	Validar corpo/erros sem persistir.
Usuário sem autenticação	401 Unauthorized	Enviar sem credencial válida.
Usuário sem autorização	403 Forbidden	Testar recurso de outro titular.
Recurso não encontrado	404 Not Found	Consultar identificador inexistente.
Estado de negócio conflitante	409 Conflict	Reservar horário já ocupado.
Limitação de tráfego	429 Too Many Requests	Verificar políticas e Retry-After se usado.
Erro inesperado	500 Internal Server Error	Verificar não vazamento de stack trace.

Contratos também têm propriedades não funcionais

- Definir timeout, paginação máxima, limites de tamanho e política de retry.
- Documentar comportamento idempotente de PUT/DELETE e identificar onde POST exige chave idempotente.
- Separar validação sintática (DTO) de regra de negócio (domínio).
- Definir política de versionamento e compatibilidade com consumidores.

Desafio: escreva a especificação OpenAPI de uma API de reservas com criação, consulta, cancelamento, filtros, erros previsíveis e autenticação. Depois implemente testes de contrato com requisições HTTP.

LABORATÓRIO PRÁTICO

PostgreSQL + Flyway · laboratório guiado

Migração reproduzível e verificação explícita do esquema.

Faça uma migração versionada antes de integrar Hibernate. O schema deve existir por migração, não como efeito colateral de `ddl-auto=create` [R29].

Dependências e arquivo de migração

- No projeto Spring Boot 4, inclua `spring-boot-starter-flyway` e `org.flywaydb:flyway-database-postgresql`, além de JDBC/JPA e driver PostgreSQL.
- Crie o arquivo `src/main/resources/db/migration/V1__criar_tarefas.sql` e configure a conexão por variáveis de ambiente.
- Em ambiente de aprendizagem, use `spring.jpa.hibernate.ddl-auto=validate` para detectar divergências entre entidades e schema.

```
CREATE TABLE tarefa (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    titulo VARCHAR(120) NOT NULL,
    concluida BOOLEAN NOT NULL DEFAULT FALSE,
    criada_em TIMESTAMP WITH TIME ZONE
        NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_tarefa_concluida
    ON tarefa (concluida);
```

Experimentos de banco

- Execute a aplicação duas vezes e verifique que a migração não é reaplicada como script novo.
- Crie V2__adicionar_prioridade.sql com coluna e default e comprove sua compatibilidade com dados existentes.
- Tente inserir título nulo e identifique qual restrição impede a operação.
- Compare plano de execução com `EXPLAIN (ANALYZE, BUFFERS)` para consultas com e sem filtro, usando massa de teste suficiente.
- Discuta por que migrations publicadas geralmente pedem uma nova versão corretiva, em vez de edição retroativa.

> **Cuidado:** uma transação que faz rollback não equivale a um mecanismo automático de desfazer migrações já implantadas; planeje evolução de schema compatível com deploy.

LABORATÓRIO PRÁTICO

Testes Spring Boot 4 · prática correta

Hierarquia de testes, diferenças de versão e critérios verificáveis.

Não trate todos os testes como `@SpringBootTest`. Escolha a menor camada que demonstre o comportamento e acrescente integração real para contratos, transações e persistência.

Tipo	O que isolar / integrar	Ferramenta
Unidade	Regras de domínio sem contexto Spring	JUnit + AssertJ; Mockito só se necessário.
MVC slice	Status, DTOs, serialização e validação HTTP	@WebMvcTest + MockMvc; dependência webmvc-test.
Integração completa	Contexto Spring e requisição HTTP	@SpringBootTest + @AutoConfigureMockMvc.
Persistência	SQL, migrations, índices e transações	Testcontainers PostgreSQL + @ServiceConnection.
Segurança	Acesso anônimo, autenticado e acesso indevido	spring-boot-starter-security-test.

Atenção à mudança da série 4

```
@SpringBootTest
@AutoConfigureMockMvc
class ApiIntegrationTest {
    @Autowired MockMvc mvc;

    // Acrescente teste HTTP e assertions do contrato.
}
```

Esqueleto intencional: requer imports, starters e classe de aplicação. Em Boot 4, `@SpringBootTest` por si só não injeta suporte ao MockMvc [R28].

Sequência de avaliação

- Escreva teste que falha, implemente o comportamento, refatore e repita.
- Teste simultaneidade e rollback no PostgreSQL real; um teste unitário não substitui concorrência real.
- Verifique se um usuário A consegue acessar o objeto do usuário B; esse teste detecta falhas horizontais frequentes.
- Não use cobertura percentual isolada como definição de qualidade.

> **Prova:** uma regressão deve ser reproduzida por um teste vermelho e corrigida até que o teste fique verde, sem comprometer outros fluxos.

LABORATÓRIO PRÁTICO

Spring Security • autenticação sem atalhos

OAuth2/OIDC, sessão, JWT e autorização de objeto.

Autenticar um usuário não significa autorizá-lo a qualquer recurso. Aprenda a separar identidade, credencial, permissão e acesso a cada objeto [R30, R31].

Duas arquiteturas para estudar

Cenário	Modelo	Atenção crítica
Aplicação web com sessão	Cookie de sessão e Spring Security	CSRF e atributos Secure/HttpOnly/SameSite.
API protegida por IdP	OAuth2 Resource Server com JWT	Validar assinatura, iss, aud, exp/nbf e escopos.
Chamadas entre serviços	OAuth2 client credentials ou mTLS	Gestão de segredos, identidade da carga e rotação.

Configuração de referência, não credenciais reais

```
# application.yml: exemplo com servidor de identidade
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://idp.exemplo.invalid/issuer
          audiences: api-backend
```

A URL acima é apenas ilustrativa. Configure um provedor de identidade de teste real para executar a aplicação. Use starter `spring-boot-starter-security-oauth2-resource-server` e as configurações oficiais [R30].

Exercícios de segurança

- Comparar 401 (não autenticado) e 403 (autenticado, porém sem permissão).
- Implementar regra “somente proprietário pode editar” e testar troca de IDs na URL (BOLA/IDOR).
- Investigar riscos de desativar CSRF em aplicações com cookies; não adotar uma regra genérica.
- Definir política de redaction em logs, limites de requisição e expiração/rotação de credenciais.

> **Regra:** não desenvolva emissão própria de JWT como atalho para aprender autenticação moderna. Priorize compreender padrões e validar tokens com bibliotecas mantidas.

LABORATÓRIO PRÁTICO

Diagnóstico de incidentes e performance

Da hipótese à evidência; meça antes de otimizar.

Desempenho backend deve ser investigado com evidências: latência, erro, saturação, banco e JVM. A otimização vem depois de reproduzir o problema.

Sintoma	Hipóteses prioritárias	Evidência a coletar
API lenta ao listar	N+1, query sem índice, paginação ausente	EXPLAIN ANALYZE, contagem SQL, p95.
Timeout intermitente	Pool JDBC saturado, dependência externa lenta	Conexões ativas, timeouts, tracing.
CPU alta	Loop, serialização, algoritmo, GC	JFR, flamegraph/profiler, GC logs.
Memória crescente	Cache sem limite, retenção, leak	Heap dump, histograma, métricas.
Erros sob carga	Race condition, limites, retries excessivos	Reprodução concorrente, logs com correlação.

Laboratório de investigação

- Crie dados de teste suficientes e defina ambiente, concorrência e duração do benchmark.
- Meça baseline: taxa de requisições, p50/p95/p99, erros, CPU e uso de memória.
- Aplique uma alteração por vez: índice, query, cache ou tamanho de pool.
- Rode exatamente o mesmo cenário de carga e compare; registre resultados e regressões.
- Escreva um post-mortem curto: impacto, linha do tempo, causa, mitigação e prevenção.

Material de estudo: Actuator e Micrometer para observabilidade, JDK Flight Recorder para profiling. As features de tracing requerem instrumentação/exportação adequada; adicionar o Actuator não cria automaticamente um stack completo de observabilidade [R32, R33].

Critério: apresentar números reproduzíveis antes/depois, delimitar incertezas e explicar por que a otimização foi escolhida.

LABORATÓRIO AVANÇADO

Backends confiáveis • falhas e consistência

Falhas controladas, testes concorrentes e idempotência.

Projetos avançados precisam comportar-se corretamente diante de concorrência, indisponibilidade e mensagens duplicadas. Não confunda “funcionou uma vez” com confiabilidade.

Propriedade	Experimento obrigatório	Resultado verificável
Atomicidade	Falhar entre duas operações SQL na mesma transação	Nenhum estado parcial persistido.
Concorrência	Disparar duas reservas para o mesmo horário	Invariante garantida por constraint/lock adequado.
Idempotência	Repetir requisição com mesma chave	Sem criação dupla do efeito de negócio.
Outbox	Falhar após commit e antes do envio de evento	Evento registrado para reprocessamento.
Consumidor robusto	Entregar duas vezes a mesma mensagem	Consumo idempotente sem duplicar efeitos.
Retry e timeout	Simular serviço externo lento/indisponível	Tentativas limitadas e política documentada.

Questões para arquitetura

- Quais invariantes pertencem ao banco e quais pertencem ao domínio?
- Quando locks pessimistas, versão otimista ou restrições únicas são apropriados?
- Qual comportamento aceito se um serviço externo estiver indisponível?
- O que a mensageria resolve que uma chamada síncrona não resolve?
- Onde estão os riscos de perda, duplicação e reordenação de mensagens?

> **Importante:** nenhuma tecnologia elimina trade-offs. Faça primeiro um monólito modular com transações claras; introduza sistemas distribuídos somente com requisitos, métricas e capacidade operacional.

ANEXO DE PROVAS

Casos de teste de referência

Critérios verificáveis, independentes do julgamento subjetivo do aluno.

Conjunto de cenários por domínio

ID	Ação	Resultado esperado
T01	Criar reserva em horário livre	201 com ID e localização, sem duplicatas.
T02	Criar reserva simultânea no mesmo recurso/horário	No máximo uma reserva confirmada.
T03	Repetir POST com a mesma Idempotency-Key	Um único efeito; resposta contratada consistente.
T04	Tentar acessar reserva de outro tenant	Negado; nenhum dado privado na resposta.
T05	Retirar permissão de usuário autenticado	403 para ação proibida; 401 só quando falta autenticação.
T06	Falhar após primeiro update da transferência	Rollback sem saldo parcial.
T07	Enviar evento duas vezes	Um único efeito externo/estado final idempotente.
T08	Simular API externa lenta	Timeout observável, retries limitados, fila/pool protegidos.
T09	Reiniciar consumer após crash	Evento pendente recuperado, sem duplicidade de efeito.
T10	Rodar query com e sem índice	Plano e tempo medidos, premissas documentadas.

Modelo de evidência para cada teste

Setup: dados e configuração. **Ação:** payload/comando. **Assert:** resultado esperado. **Instrumentação:** logs/SQL/metricas. **Reprodução:** comando único no README.

> **Regra:** uma aprovação exige teste executável, mais a explicação escrita do mecanismo que garante o comportamento.

Estudo em hardware limitado • plano leve

O estudo não exige rodar toda a infraestrutura avançada desde o primeiro mês.

Java, IDE, navegador, banco e containers competem por memória. Para aprender de forma consistente, configure o ambiente de acordo com os recursos disponíveis.

Se seu computador tem cerca de 8 GB de RAM

- Nos meses 1–6, prefira Java, Maven, editor/IDE e PostgreSQL sem Docker rodando o dia inteiro.
- Use o Maven Wrapper e feche programas pesados durante builds. Reinicie apenas processos travados, em vez de “otimizadores mágicos”.
- Rode containers e testes Testcontainers sob demanda. Defina recursos prudentes para o runtime Docker/WSL e avalie o consumo com ferramentas do sistema.
- Em laboratórios de performance, monitore pressão de memória e swaps; resultados medidos em notebook podem não representar um servidor.
- Não execute simultaneamente múltiplos bancos, Kafka, Redis, tracing e IDE se o hardware não comportar; distribua o laboratório em etapas.

Alternativa progressiva

Etapa	Ambiente mínimo
Java inicial	JDK + IDE + Git + Maven Wrapper.
SQL	PostgreSQL local + psql ou DBeaver.
Spring REST	API local + testes unitários.
Testes com banco	Docker/Testcontainers somente na sessão necessária.
Sistemas distribuídos	Uma dependência por vez; CI ou máquina externa para carga pesada.

Checklist de saúde: versão do JDK (`java -version`), build (`.\mvnw.cmd -version`), teste (`.\mvnw.cmd test`), banco (conexão e migração), memória e espaço livre.

AVALIAÇÃO PRÁTICA

Matriz de checkpoints por nível

Critérios observáveis por fase e método de diagnóstico técnico.

Um currículo técnico só é efetivo quando as entregas demonstram autonomia crescente. Use esta matriz para decidir quando avançar, independentemente da quantidade de meses.

Marco	Prova prática sem tutorial	Evidência
Mês 3 · Java	Implementar CLI com validação, OOP, coleções e testes	Git + README + testes verdes + explicação.
Mês 6 · Base backend	Depurar Java, consultar banco e explicar HTTP	SQL real + logs + comandos reproduzíveis.
Mês 9 · Spring Core	Criar API mínima, explicar beans e DI	Endpoint executável e teste HTTP.
Mês 12 · API profissional	Criar API persistente com controle de acesso por objeto	Migrations, testes e casos de erro.
Mês 15 · Entrega	Testar e empacotar em CI/CD com rollback documentado	Pipeline + runbook + artefato.
Mês 18 · Operação	Investigar falha/latência induzida com observabilidade	Métricas, logs e relatório de causa.
Mês 21 · Arquitetura	Defender desenho de um sistema sob restrições	ADR + comparação de alternativas.
Mês 24 · Capstone	Demonstrar resiliência, segurança e performance verificáveis	Projeto reproduzível, testes de carga e apresentação.

Como pontuar cada eixo (0–4)

- **0:** ainda não reconhece o conceito. **1:** reconhece, mas não aplica.
- **2:** consegue executar com tutorial ou ajuda pontual.
- **3:** implementa e explica sem guia, incluindo testes e erros.
- **4:** aplica a caso novo, compara alternativas e avalia trade-offs.

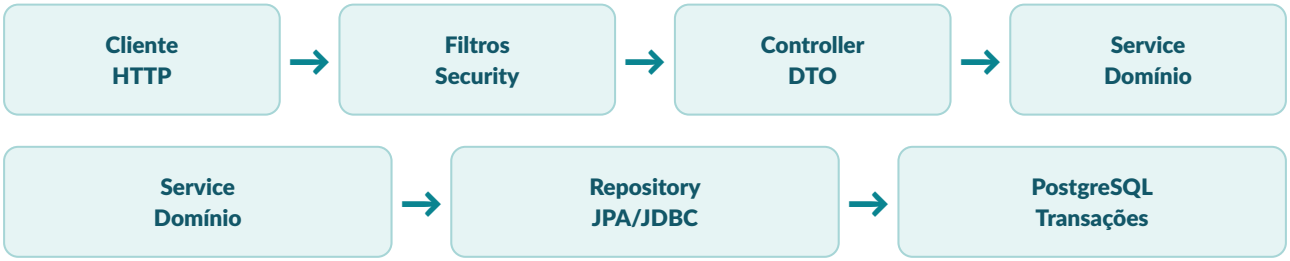
> **Regra de avanço:** obtenha pelo menos 3 nos eixos compreensão, implementação, depuração e validação; os conteúdos avançados continuam sendo revisitados em projetos posteriores.

ARQUITETURA

Fluxo de uma requisição Spring

Responsabilidade, coesão e fluxo de dados.

O desenho abaixo representa um fluxo típico de API síncrona. As responsabilidades são mais importantes do que a quantidade de classes.



Elemento	Responsabilidade	Evite
Controller	Traduzir HTTP para caso de uso e resposta	Executar queries e regras complexas.
Service / caso de uso	Orquestrar invariantes e fronteiras transacionais	Depender de detalhes de protocolo sem necessidade.
Domínio	Representar regras e estado válido	Referenciar Spring sem benefício real.
Repository	Acessar dados preservando contrato	Expor implementações ORM a todas as camadas.
Infraestrutura	Configuração, mensageria e provedores externos	Contaminar modelos de domínio com SDKs.

Exemplo de organização por feature

```
br.com.exemplo.pedidos/
  PedidoApplication.java
  pedidos/
    api/            # HTTP, DTOs, validação
    application/    # Casos de uso
    domain/         # Regras e modelos
    persistence/    # JPA / SQL
  catalogo/
    api/ application/ domain/ persistence/
  shared/
    config/         # Compartilhado e realmente transversal
```

> **Princípio:** a arquitetura deve facilitar mudança e testes. A divisão exata depende do domínio; não crie camadas vazias apenas para imitar um diagrama.

ARQUITETURA

Quando crescer: decisões arquiteturais

Use dados, requisitos e riscos para decidir.

Sequência de evolução recomendada

Problema observado	Primeira hipótese	Validar com
Endpoint lento	SQL, N+1, índices e cache local	EXPLAIN, profiling, p95, query count
Processamento demorado	Job assíncrono e desacoplamento	Tempo de resposta, fila, taxa de falhas
Escala de leitura	Cache, réplicas e páginas menores	Throughput, consistência, hit rate
Domínios muito acoplados	Modularização e contratos internos	Mudanças entre módulos e testes
Falha entre gravação e evento	Outbox e consumidor idempotente	Simulação de crash/duplicidade
Equipe precisa evoluir isoladamente	Avaliar serviço separado só após modularizar	Custo operacional vs autonomia obtida

Template compacto de ADR

ADR-0001 • Decisão técnica

Contexto: problema real, requisitos e restrições.
Alternativas: opções consideradas com custos.
Decisão: escolha explícita e justificativa.
Consequências: benefícios, riscos, migração e reversão.
Evidências: métricas, testes e fontes que sustentam a escolha.

Perguntas obrigatórias antes de microsserviços

- O monólito modular já está bem delimitado e tem gargalo comprovado?
- Há necessidade real de independência de deploy e operação?
- O time domina tracing, observabilidade, contratos e transações distribuídas?
- Qual será a estratégia para dados, versionamento, falhas e consistência?
- Qual é o custo operacional total versus o ganho esperado?

> **Avançado não é sinônimo de distribuído.** Um monólito bem organizado pode ser a melhor arquitetura para muitos produtos e equipes.

AVALIAÇÃO

Rubrica de avaliação trimestral

Critérios observáveis para diminuir a ilusão de competência.

Use a rubrica abaixo com notas de 0 a 4. Só avance em uma área crítica quando atingir pelo menos 3 de forma consistente.

Nota	Evidência observável
0 · Não conheço	Não reconhece o conceito, mesmo com exemplos.
1 · Reconheço	Consegue explicar uma definição, mas não aplicar.
2 · Faço com apoio	Executa com documentação e auxílio intenso.
3 · Faço sozinho	Resolve problema novo, testa e explica decisões.
4 · Avalio e ensino	Compara alternativas, identifica limites, adapta e revisa outros códigos.

Matriz de revisão (preencher a cada trimestre)

Dimensão	0–4	Evidência ou link
Lógica e Java	____	_____
Modelagem e SQL	____	_____
Spring / APIs	____	_____
Segurança	____	_____
Testes e debugging	____	_____
Git / Deploy / Operação	____	_____
Arquitetura / trade-offs	____	_____

> **Critério para uma fase concluída:** projeto executável + testes relevantes + README + demonstração de autonomia + explicação escrita das decisões. Refaça competências abaixo de 3.

Registro de estudo

Data: _____ **O que estudei:** _____

O que implementei: _____

O que ainda não consigo explicar: _____

Próximo experimento: _____

CARREIRA

Entrevistas, portfólio e mercado

O objetivo é demonstrar competência em problemas reais.

Domínios para revisão técnica

Área	Questões que você deve conseguir explicar
Java	equals/hashCode, collections, exceptions, generics, Streams, GC, concorrência e imutabilidade.
Spring	IoC/DI, beans, MVC, filtros, validação, AOP/proxies, transações e configuração.
SQL/JPA	JOIN, índices, isolamento, N+1, lazy loading, migrations e consultas lentas.
Web	HTTP, autenticação, autorização, TLS, cookies, REST, idempotência e caching.
Qualidade	Testes por camada, mocks, integração, design, refatoração e diagnóstico.
Arquitetura	Capacidade, falhas, escalabilidade, consistência, filas e decisão de boundaries.

Portfólio de 3 projetos em destaque

- **Projeto com domínio e SQL:** API de biblioteca ou reservas, com regras difíceis e testes reais.
- **Projeto com segurança e produção:** API autenticada, CI/CD, deploy, logs e documentação.
- **Projeto avançado:** sistema de pedidos com concorrência, eventos, idempotência e métricas.

Simulação de entrevista prática

- Desenhe uma API para reservas com prevenção de conflitos.
- Explique por que uma requisição retornou 500 e como investigaria.
- Mostre um teste que falhava e que passou depois da correção.
- Compare usar transação no banco versus apenas lógica Java.
- Desenhe a evolução de uma API com aumento de carga de 10 vezes.

Avalie decisões e clareza, não apenas memória. Experiência profissional depende também de colaboração, manutenção de sistemas existentes, comunicação e responsabilidade por incidentes.

PRÁTICA DE CÓDIGO REAL

Como estudar repositórios de referência

Leia código de projetos maduros sem copiar padrões fora de contexto.

Seis repositórios para estudar

Repositório	Foco principal
spring-petclinic	Aplicação Spring pequena e testes.
spring-modulith	Fronteiras de módulos e eventos.
thombergs/buckpal	Arquitetura hexagonal aplicada.
ddd-by-examples/library	DDD, eventos e invariantes.
Rinha de Backend	Performance, concorrência e editais históricos.
gothinkster/realworld	Contrato de API e comparação de implementações.

Sessão estruturada de 60–90 minutos

- Execute os testes e identifique as dependências de build.
- Siga uma requisição do endpoint até a persistência.
- Escreva cinco decisões que você manteria ou mudaria, com motivo.
- Modifique uma regra de negócio e observe quais testes falham.
- Registre uma descoberta no caderno de engenharia e o link do commit/PR.

> Repositórios são retratos de versões diferentes; não presuma compatibilidade automática com Boot 4.1 e Java 25. Confira licença, branches e dependências.

Biblioteca de referências • I

Links clicáveis para documentação oficial e recursos de apoio.

Priorize documentação oficial. Os links abaixo foram selecionados para consulta ao longo das etapas; confirme a versão exata ao iniciar cada projeto.

[R01] Java SE Support Roadmap (Oracle)
Versões LTS e política de suporte.
https://www.oracle.com/br/java/technologies/java-se-support-roadmap.html

[R02] Eclipse Temurin / Adoptium
Instalação do JDK no Windows e outros sistemas.
https://adoptium.net/pt-BR/installation

[R03] Spring Boot Reference
Configuração, starters, Actuator e operação.
https://docs.spring.io/spring-boot/reference/

[R04] Spring Boot - requisitos
Compatibilidade de Java e ferramentas.
https://docs.spring.io/spring-boot/4.0/system-requirements.html

[R05] Spring Initializr
Criação de projetos Java com dependências suportadas.
https://start.spring.io/

[R06] Spring Framework Reference
IoC, MVC, transações e Spring Core.
https://docs.spring.io/spring-framework/reference/

[R07] Spring Guides
Projetos e tutoriais oficiais por área.
https://spring.io/guides

[R08] Spring Academy
Cursos organizados do ecossistema Spring.
https://spring.academy/courses

[R09] Spring Data JPA
Repositories, consultas, transactions e locking.
https://docs.spring.io/spring-data/jpa/reference/

[R10] Spring Security Reference
Autenticação, autorização e segurança.
https://docs.spring.io/spring-security/reference/

[R11] Spring Boot 4 Migration Guide
Diferenças e mudanças de starters na série Boot 4.
https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-4.0-Migration-Guide

[R12] PostgreSQL Tutorial
SQL, joins, modelagem e transações.
https://www.postgresql.org/docs/current/tutorial.html

[R13] OWASP Top 10:2025 em português
Riscos e práticas de segurança atualizados.
https://top10.owasp.org/2025/pt-BR/

REFERÊNCIAS E DOCUMENTAÇÃO

Biblioteca de referências • II

Links clicáveis para documentação oficial e recursos de apoio.

Priorize documentação oficial. Os links abaixo foram selecionados para consulta ao longo das etapas; confirme a versão exata ao iniciar cada projeto.

[R14] JUnit User Guide
Testes no ecossistema Java.
https://docs.junit.org/current/user-guide/

[R15] Testcontainers Java
Testes com dependências reais em containers.
https://java.testcontainers.org/

[R16] Maven Introduction
Build e gerenciamento de dependências.
https://maven.apache.org/guides/getting-started/

[R17] Pro Git
Git do básico a fluxos avançados.
https://git-scm.com/book/pt-br/v2

[R18] MDN HTTP
Conceitos e semântica HTTP.
https://developer.mozilla.org/pt-BR/docs/Web/HTTP

[R19] Docker Docs
Containers e imagens.
https://docs.docker.com/

[R20] GitHub Actions Docs
Pipelines de automação.
https://docs.github.com/pt/actions

[R21] Microsoft Learn - Java
Guias e fundamentos para Java.
https://learn.microsoft.com/pt-br/java/

[R22] OpenJDK - Java Language
Aprender Java com guias oficiais.
https://dev.java/learn/

[R23] Redis Docs
Cache e armazenamento chave-valor.
https://redis.io/docs/latest/

[R24] RabbitMQ Tutorials
Mensageria e processamento assíncrono.
https://www.rabbitmq.com/tutorials

[R25] OpenTelemetry Docs
Tracing, métricas e instrumentação.
https://opentelemetry.io/docs/

[R26] roadmap.sh Backend
Mapa complementar para revisão de lacunas.
https://roadmap.sh/backend

REFERÊNCIAS DE REVISÃO

Biblioteca de referências • III

Fontes adicionais para reproduzir e atualizar os laboratórios.

Fontes oficiais adicionadas na revisão 2026. As referências de versão são específicas da linha indicada; verifique atualizações de patch ao executar os exemplos.

[R27] Spring Boot 4 - catálogo de starters
Starters webmvc, testes específicos, BOM e dependências.
https://docs.spring.io/spring-boot/4.0/reference/using/build-systems.html

[R28] Spring Boot 4 - guia de migração
Mudanças de tests, MockMvc, modularização e Jackson 3.
https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-4.0-Migration-Guide

[R29] Spring Boot - Flyway e inicialização de dados
Migrações, starter Flyway e módulo PostgreSQL.
https://docs.spring.io/spring-boot/how-to/data-initialization.html

[R30] Spring Security - OAuth2 Resource Server JWT
Validação issuer, audience, assinatura e chaves.
https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html

[R31] OWASP API Security Top 10
BOLA, autenticação e controles próprios de APIs.
https://owasp.org/API-Security/editions/2023/en/0x11-t10/

[R32] Spring Boot - Observability
Logs, métricas, tracing e Micrometer.
https://docs.spring.io/spring-boot/4.0/reference/actuator/observability.html

[R33] Spring Boot - Actuator endpoints
Exposição segura e health checks.
https://docs.spring.io/spring-boot/4.0/reference/actuator/endpoints.html

[R34] Spring Boot - Testcontainers
ServiceConnection e integração com containers.
https://docs.spring.io/spring-boot/4.0/reference/testing/testcontainers.html

[R35] Spring Framework - @Transactional
Proxy, self-invocation e regras padrão de rollback.
https://docs.spring.io/spring-framework/reference/data-access/transaction/declarative/annotations.html

[R36] Spring Security - migração para 7
Compatibilidade e transição para Spring Security 7.
https://docs.spring.io/spring-security/reference/7.0/migration/

[R37] HTTP Problem Details - RFC 9457
Contrato interoperável de erros em APIs HTTP.
https://www.rfc-editor.org/rfc/rfc9457

BIBLIOTECA CURADA

Livros: um por fase

Combine teoria rigorosa com código executável. Priorize leitura seletiva associada ao projeto.

Sequência sugerida

Fase	Livro	Foco
Java	Effective Java (3ª ed.) — Joshua Bloch	APIs, imutabilidade, generics e object contracts.
Concorrência	Java Concurrency in Practice — Brian Goetz e outros	Memória, visibilidade, locks (fundamentos duradouros).
Persistência	High-Performance Java Persistence — Vlad Mihalcea	JDBC, Hibernate, locking e transações.
Segurança	Spring Security in Action (2ª ed.) — Laurențiu Spilcă	Filtros, autenticação, OAuth2.
Domínio	Learning Domain-Driven Design — Vlad Khononov	Estratégia/tática e limites do DDD.
Arquitetura	Get Your Hands Dirty on Clean Architecture — Tom Hombergs	Hexagonal executável.
Distribuídos	Designing Data-Intensive Applications (2ª ed., 2026) — Kleppmann/Riccomini	Sistemas de dados, replicação, consistência.
Operação	Release It! (2ª ed.) — Michael Nygard	Padrões de confiabilidade.

Edições e exemplos de bibliotecas refletem suas épocas; compare APIs e práticas com a documentação atual do Spring e do JDK.

> **Prática:** para cada capítulo, extraia uma decisão técnica; aplique-a em um microexperimento e escreva quando ela não deve ser utilizada.

REFERÊNCIAS COMPLEMENTARES

Fontes oficiais e guias técnicos

As URLs abaixo complementam as 37 referências do guia original.

Plataforma e Spring

- dev.java/learn — Java moderno e fundamentos.
- OpenJDK JEP Index — status de recursos de linguagem.
- Spring Blog — versões e releases.
- Boot 4 Migration Guide — compatibilidade.
- Spring Resilience — retry/limite de concorrência.
- Spring Academy — cursos oficiais.

Dados, qualidade e operação

- PostgreSQL RLS — políticas de linha.
- Use The Index, Luke — índices.
- ArchUnit — arquitetura executável.
- PIT — teste de mutação.
- Testcontainers Guides — bancos reais em testes.
- Google SRE Books — confiabilidade.

Segurança e projetos

- OWASP Top 10:2025 e Cheat Sheets.
- Spring AI — especialização *opcional* após a base.
- Release Spring Boot 4.1.2 — confirmação da baseline.
- Spring Boot 4.2.0-RC1 — pré-lançamento; não base estável em 02/11/2026.

Proveniência: complementos priorizados a partir do arquivo “Revisão e Complemento — Plano Backend Java + Spring”; detalhes de versão contrastados com fontes oficiais em 02/11/2026.

GLOSSÁRIO

Glossário essencial de backend

Termos de referência para os primeiros estudos e para revisões.

Use este glossário enquanto estuda. A definição é propositalmente curta: os módulos aprofundam mecanismos, limites e aplicações.

Termo	Definição de trabalho
JDK / JVM	JDK é o kit de desenvolvimento; JVM executa bytecode Java.
IoC / DI	Inversão de controle e injeção de dependência; separam criação de objetos do uso.
Bean	Objeto gerenciado pelo contêiner do Spring.
DTO	Objeto de transferência de dados; define contratos de entrada e saída.
ORM / JPA	Mapeamento objeto-relacional e especificação de persistência Java.
JDBC	API Java de acesso a bancos relacionais por SQL.
Transação	Unidade de trabalho com propriedades de atomicidade e consistência.
Idempotência	Repetir operação não produz efeitos adicionais além do primeiro efeito previsto.
N+1	Padrão de acesso que dispara consultas adicionais em série para uma lista.
JWT / OAuth2	JWT é formato de token; OAuth2 é arcabouço de autorização, não sinônimos.
CI/CD	Automação de integração, testes e processos de entrega.
RPS / p95	Requisições por segundo / latência abaixo da qual caem 95% das amostras.
SLI / SLO	Indicador medido e objetivo definido para confiabilidade de serviço.
DDD	Modelagem orientada ao domínio e a seus limites organizacionais.
Outbox	Persistência atômica de dados de negócio e registro de evento para entrega posterior.
DLQ	Fila para mensagens que não foram processadas após políticas de tratamento.
ADR	Registro curto de decisão arquitetural, contexto e consequências.
CAP	Trade-off sobre consistência e disponibilidade perante partições de rede.

Como usar: sempre que um termo aparecer pela primeira vez, tente explicá-lo com um exemplo concreto e um contraexemplo. Memorizar siglas não substitui compreender o mecanismo.

CONTINUIDADE

Checklist final • sua trilha pessoal

O verdadeiro indicador de progresso é o que você consegue construir e explicar.

Use esta página como um registro de entregas. Marque somente quando existir evidência concreta de cada habilidade.

Marcar	Habilidade demonstrada
☐	Escrevi Java básico e tratei entradas inválidas.
☐	Modelei classes, interfaces e regras de domínio sem Spring.
☐	Escrevi testes automatizados e executei com Maven.
☐	Utilizei Git com branches, merges e revisão de código.
☐	Projetei e consultei um banco PostgreSQL usando SQL.
☐	Expliquei HTTP, TLS, status e idempotência.
☐	Construí API com Spring MVC, DTOs e validação.
☐	Usei JPA com transações e investi em queries eficientes.
☐	Implementei autenticação e autorização testadas.
☐	Criei testes unitários e de integração com banco real.
☐	Empacotei, implantei e mantive um serviço com CI/CD.
☐	Investiguei desempenho com logs, métricas e tracing.
☐	Projetei filas/cache e tratei falhas ou mensagens duplicadas.
☐	Justifiquei arquitetura com hipóteses, ADRs e trade-offs.
☐	Apresentei um sistema completo e reproduzi seus resultados.

Próxima ação: reserve os próximos sete dias para configurar o ambiente, desenvolver a calculadora Java e publicar o primeiro repositório. A habilidade vem da execução repetida e da avaliação crítica.

Material revisado com fontes oficiais em 02/11/2026. O roteiro não representa certificação oficial, nem garante determinado nível de senioridade em prazo fixo. Recursos e versões devem ser verificados ao utilizá-los.

COMECE A IMPLEMENTAR

Primeiros 30 dias • plano adaptativo

Execute a trilha acelerada somente se as provas D1–D3 comprovarem domínio.

Semana 1 — Diagnóstico Java, SQL e HTTP

- Resolver D1 e D2 sem tutorial/IA e registrar tempos.
- Criar repositório `backend-java-learning` com pasta de evidências.
- Pontuar rubrica, abrir issues com lacunas e preparar sprints de recuperação.

Semana 2 — Spring Core e baseline

- Resolver D3; se falhar, estudar os mecanismos específicos.
- Gerar projeto pelo Initializr: Boot 4.1.2, Java 25, Web MVC, Validation, JPA, Flyway, PostgreSQL.
- Ler `pom.xml` e verificar versão do Maven Wrapper.

Semana 3 — Java moderno e JVM

- Modelar estados com records, sealed e pattern matching.
- Executar experimento JFR e registrar hipótese/perfil observado.
- Ler seletivamente *Effective Java* e aplicar um dos princípios em código.

Semana 4 — PostgreSQL concorrente

- Resolver consulta com índice e EXPLAIN (ANALYZE, BUFFERS).
- Implementar transferência JDBC usando locking com IDs ordenados.
- Executar Gate F1 com 50 tarefas concorrentes; validar saldos e documentar limites do pool.

> Se o diagnóstico apontar nível básico, **não** salte para JDBC concorrente: utilize o roteiro de oito semanas do guia original e retome este cronograma após concluir os gates.

> **Primeira entrega pública:** README + código + testes + relatório do diagnóstico + resultados do laboratório.

HISTÓRICO DE EDIÇÃO

O que foi integrado nesta edição

Complemento do usuário incorporado com critérios de precisão e caminho adaptativo.

Correções e ampliações realizadas

- Baseline editorial atualizada para Spring Boot 4.1.2 (02/11/2026); Boot 4.2.0-RC1 tratado como pré-lançamento.
- Trilhas de 24 e 12 meses com prova diagnóstica; nenhuma experiência prévia presumida.
- Java moderno, profiling/JFR, Spring Framework 7, internals do container e RestClient.
- PostgreSQL avançado: MVCC, locks, filas com SKIP LOCKED; multi-tenancy e RLS.
- Spring Modulith, arquitetura testável, mutação, testes de carga, SBOM e Buildpacks.
- Quatro projetos avançados com aceites, leitura de seis repositórios e biblioteca de livros.
- Correções de BigDecimal, ordem de locks e Testcontainers e casos de teste de referência.

O que foi preservado do plano original

- 24 meses, 17 módulos, cinco projetos progressivos, 20 exercícios e laboratórios básicos.
- Matriz de competências, rubrica 0–4 e gates de avanço por evidência.
- Princípio de arquiteturas incrementais: medir → hipótese → alteração → nova medição.

> **Nota editorial:** os prazos são aproximados; a trilha acelerada foi desenhada como opção condicionada às provas, não como atestado de experiência profissional.